Lecture 2

Variational Autoencoder

变分自编码器

6.S978 Deep Generative Models

Kaiming He
Fall 2024, EECS, MIT

Overview

- Variational Autoencoder (VAE)

如何通过一个包含潜变量的概率模型，学习真实数据分布，并且生成新的数据。

- Relation to Expectation-Maximization (EM)

为什么VAE的学习过程中会涉及潜变量分布和生成模型参数这两个不同的优化对像

- Vector Quantized VAE (VQ-VAE)

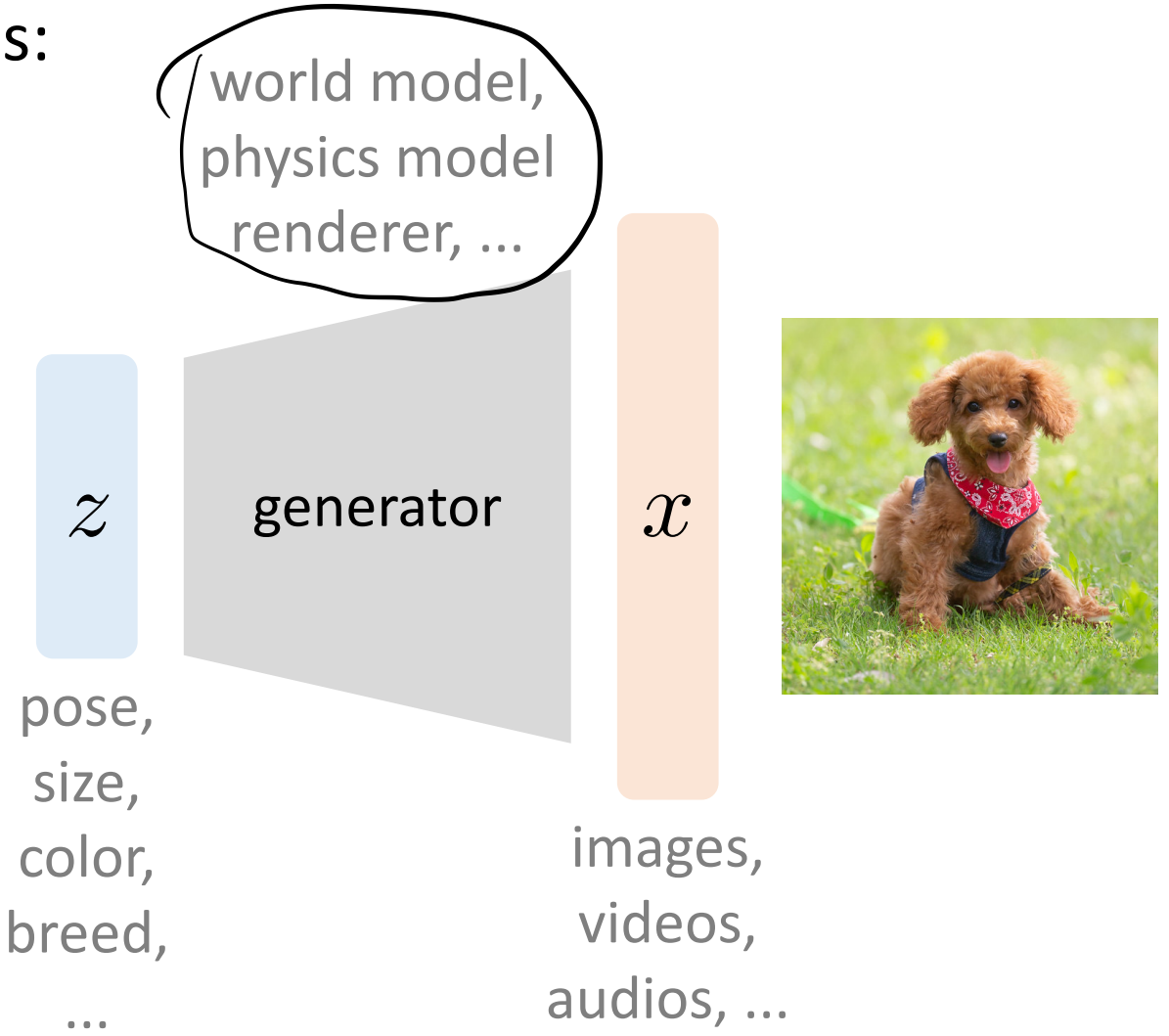
Variational Autoencoder (VAE)

Latent Variable Models

Assuming a data generation process:

- z - latent variables
- x - observed variables

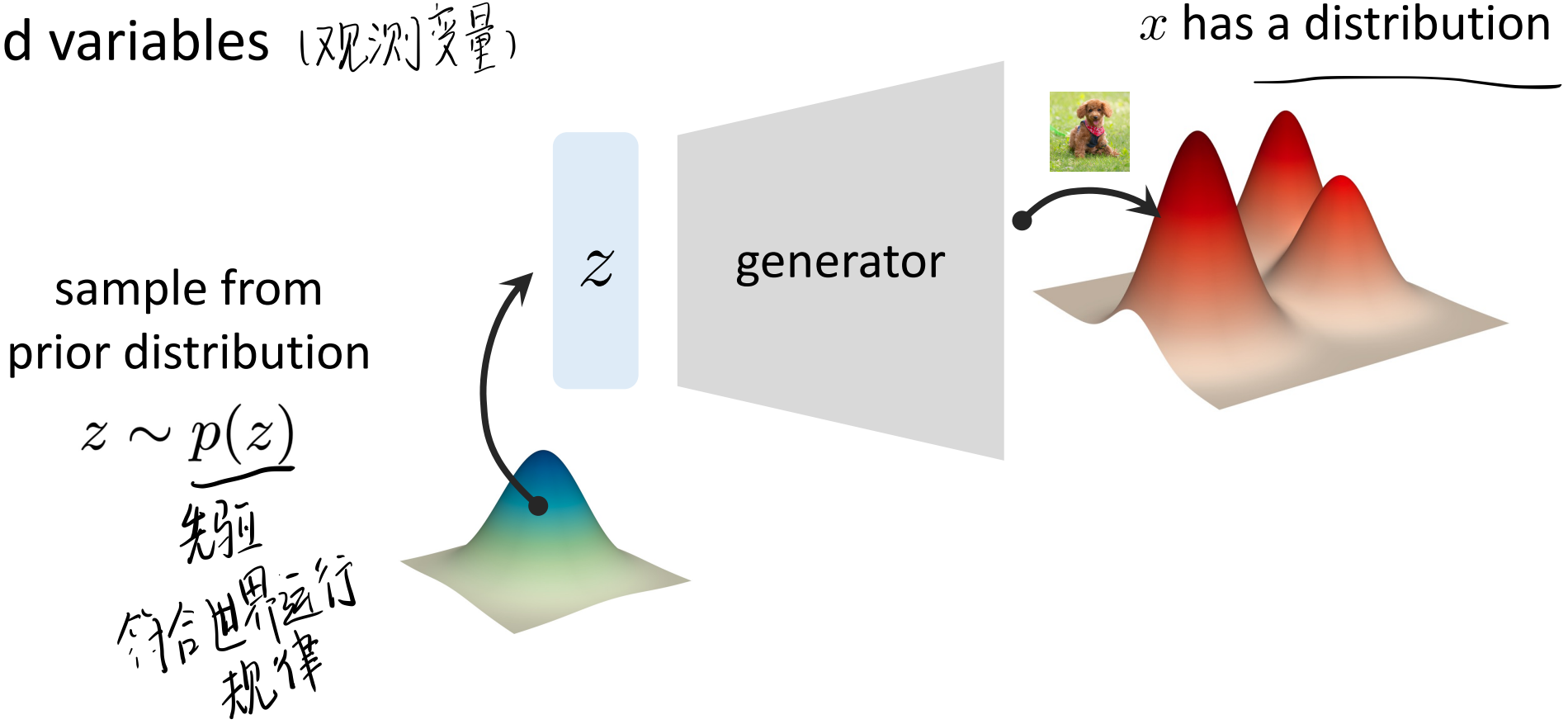
VAE：概率分布 ①具有概率分布 ②采样生成new



Latent Variable Models

Assuming a data generation process:

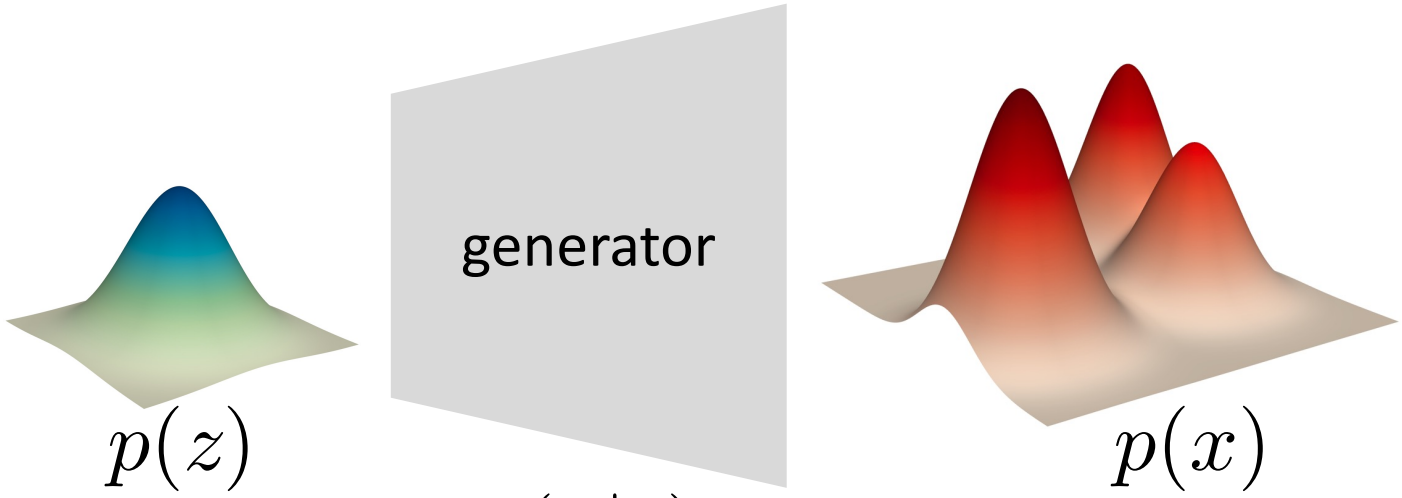
- z - latent variables (潜变量)
- x - observed variables (观测变量)



Latent Variable Models

Assuming a data generation process:

- z - latent variables
- x - observed variables



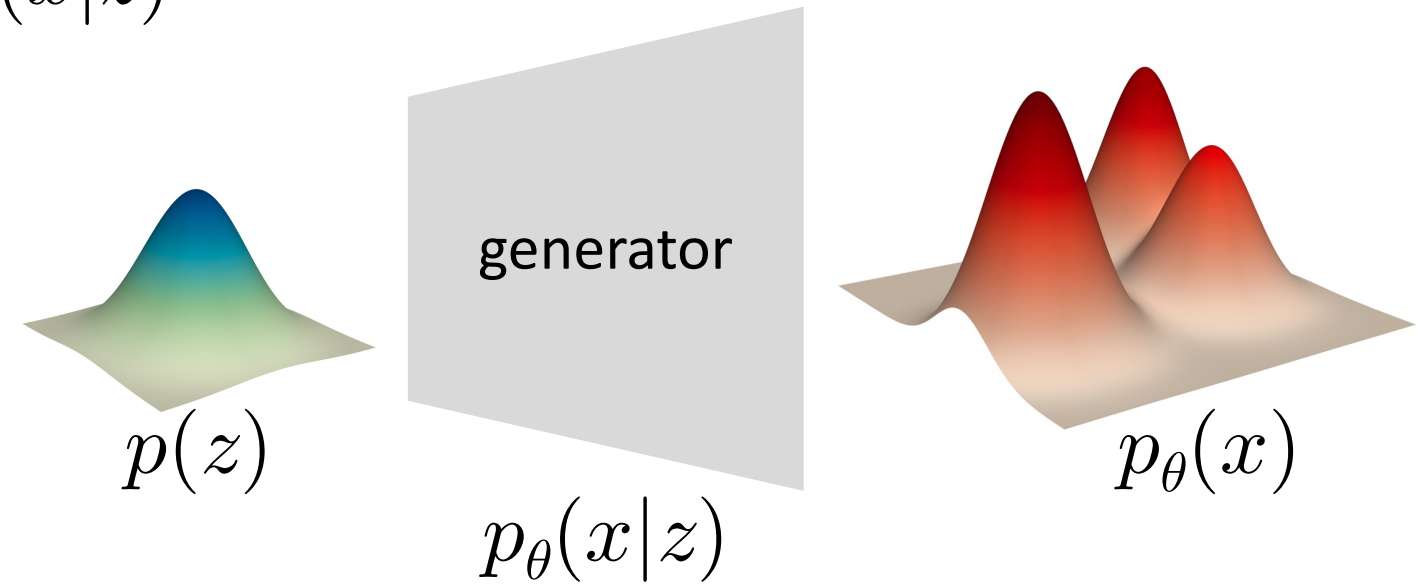
潜变量 z 的先验分布

当潜变量 z 已经确定时，观测数据 x 的条件概率分布是什么

Latent Variable Models

Represent a distribution by a neural network

- θ - learnable parameters
- represent a function: $p_\theta(x|z)$



输入 z，网络根据当前的参数 θ，给出与生成 x 有关的结果。

Measuring how good a distribution is …

Minimize Kullback–Leibler (KL) divergence:

$$\min_\theta \; \mathcal{D}_{\mathrm{KL}}(\, p_{data} \, || \, p_\theta \,)$$

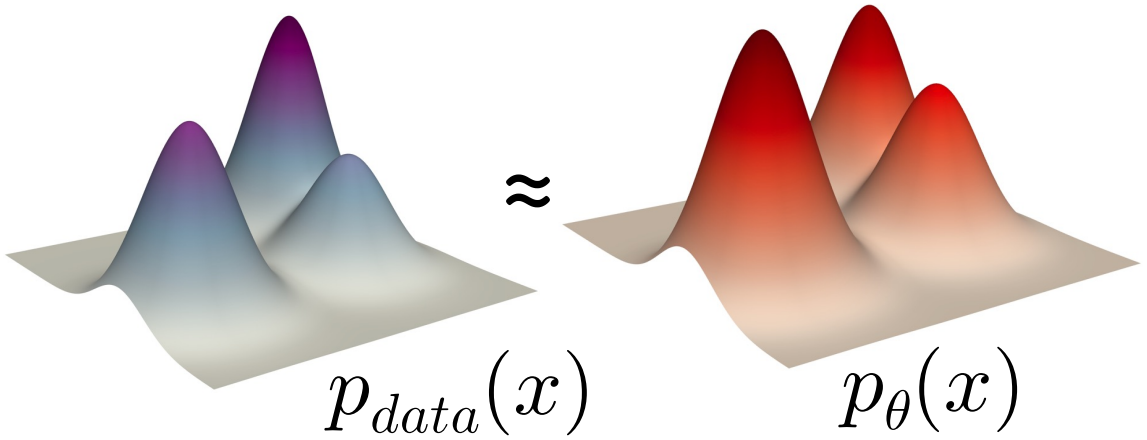
Note: consider other criteria than KL?

⇒ Maximize likelihood:

$$\max_\theta \; \mathbb{E}_{x \sim p_{data}} \log p_\theta(x)$$

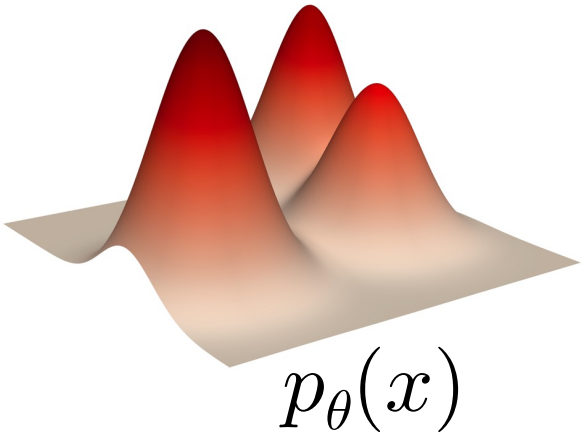
tl; dr

$$\begin{aligned}
& \arg\min_\theta \; \mathcal{D}_{\mathrm{KL}}(\, p_{data} \, || \, p_\theta \,) \\
= \; & \arg\min_\theta \; \sum_x p_{data}(x) \log \frac{p_{data}(x)}{p_\theta(x)} \\
= \; & \arg\min_\theta \; \sum_x -p_{data}(x) \log p_\theta(x) + const \\
= \; & \arg\max_\theta \; \sum_x p_{data}(x) \log p_\theta(x) \\
= \; & \arg\max_\theta \; \mathbb{E}_{x \sim p_{data}} \log p_\theta(x)
\end{aligned}$$



Latent Variable Models

We want to maximize $\mathbb{E}_{x \sim p_{data}} \log p_\theta(x)$

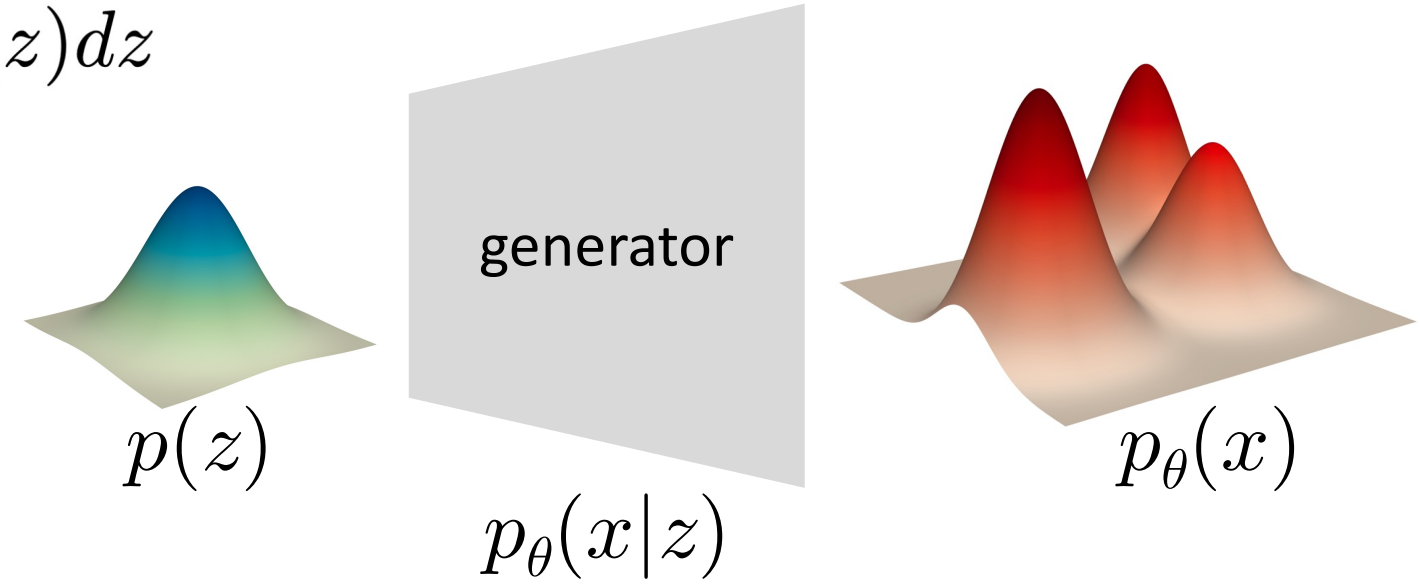


Latent Variable Models

We want to maximize $\mathbb{E}_{x \sim p_{data}} \log p_\theta(x)$

with $p_\theta(x)$ represented as:

$$p_\theta(x) = \int_z p_\theta(x|z)p(z)dz$$

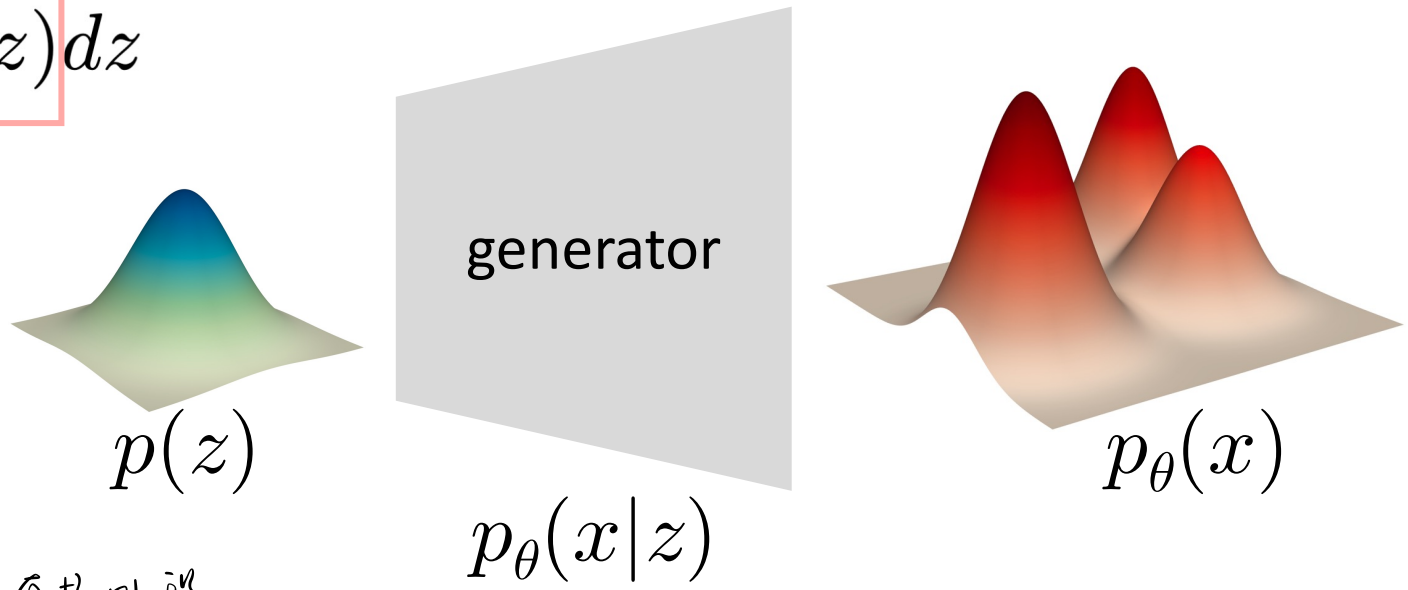


Latent Variable Models

We want to maximize $\mathbb{E}_{x \sim p_{data}} \log p_\theta(x)$

with $p_\theta(x)$ represented as:

$$p_\theta(x) = \int_z p_\theta(x|z) p(z) dz$$



Two sets of unknowns:

- We need to optimize for θ
- We can't control "**true**" $p(z)$ → 难以直接采样

Idea: introduce a "controllable" distribution $q(z)$

Latent Variable Models

$$\log p_\theta(x)$$

$$= \int_z q(z) \log p_\theta(x) dz$$

$$= \int_z q(z) \log \Big(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \Big) dz$$

$$= \int_z q(z) \log \Big(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \Big) dz$$

$$= \int_z q(z) \Big(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \Big) dz$$

$$= \mathbb{E}_{z \sim q(z)} \Big[\log p_\theta(x|z) \Big] - \mathcal{D}_{\mathrm{KL}} \Big(q(z) || p_\theta(z) \Big) + \mathcal{D}_{\mathrm{KL}} \Big(q(z) || p_\theta(z|x) \Big)$$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule

Latent Variable Models

$$\log p_\theta(x)$$

$$= \int_z q(z) \log p_\theta(x) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)$$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule
- just algebra
- just algebra

Latent Variable Models

$$\log p_\theta(x)$$

$$= \int_z q(z) \log p_\theta(x) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)$$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule
- just algebra
- just algebra

Latent Variable Models

$$\log p_\theta(x)$$

$$= \int_z q(z) \log p_\theta(x) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)$$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule
- just algebra
- just algebra

Latent Variable Models

$$\log p_\theta(x)$$

Rewrite log likelihood by latent z

$$= \int_z q(z) \log p_\theta(x) dz$$

- for any distribution $q(z)$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz$$

- Bayes' rule

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz$$

- just algebra

$$= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz$$

- just algebra

$$= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)$$

Latent Variable Models

$$\log p_\theta(x)$$

$$= \int_z q(z) \log p_\theta(x) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz$$

$$= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)$$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule
- just algebra
- just algebra

相对概率分布的比值

比值为相对熵度提供条件

Latent Variable Models

intractable $\log p_\theta(x)$

Rewrite log likelihood by latent z

- for any distribution $q(z)$
- Bayes' rule

$$
\begin{aligned}
&= \int_z q(z) \log p_\theta(x) dz \\
&= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \right) dz \\
&= \int_z q(z) \log \left(\frac{p_\theta(x|z) p_\theta(z)}{p_\theta(z|x)} \frac{q(z)}{q(z)} \right) dz \\
&= \int_z q(z) \left(\log p_\theta(x|z) + \log \frac{p_\theta(z)}{q(z)} + \log \frac{q(z)}{p_\theta(z|x)} \right) dz \\
&= \mathbb{E}_{z \sim q(z)} \left[\log p_\theta(x|z) \right] - \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z) \right) + \mathcal{D}_{\mathrm{KL}} \left(q(z) || p_\theta(z|x) \right)
\end{aligned}
$$

tractable tractable intractable

ELBO: 证据下界. VAE训练目标的数学基础.

Latent Variable Models

VAE的损失函数不是凭空.

intractable $\log p_\theta(x) - \mathcal{D}_{\mathrm{KL}}\left(q(z)||p_\theta(z|x)\right)$ intractable

$= \mathbb{E}_{z\sim q(z)}\left[\log p_\theta(x|z)\right] - \mathcal{D}_{\mathrm{KL}}\left(q(z)||p_\theta(z)\right)$

tractable tractable

第一项：如果我们能够从 q(z) 中采样，并且能够计算生成模型的条件对数概率，那么这一项就可以通过采样来近似估计。比如，我们采样十次、二十次，甚至只采样一次，再计算对应的对数概率。

第二项：如果我们把 q(z) 和 pθ(z) 都选择为一些容易处理的概率分布，例如合适的高斯分布，那么这一项就可能具有解析形式。

Latent Variable Models

intractable $\log p_\theta(x) - \mathcal{D}_{\mathrm{KL}}\left(q(z)||p_\theta(z|x)\right)$ intractable

$$= \underbrace{\mathbb{E}_{z\sim q(z)}\left[\log p_\theta(x|z)\right] - \mathcal{D}_{\mathrm{KL}}\left(q(z)||p_\theta(z)\right)}_{\text{tractable}\qquad\qquad\text{tractable}}$$

- This is called Evidence Lower Bound (ELBO)
- Lower bound of $\log p_\theta(x)$

- This equation holds for <u>any</u> distribution $q(z)$

Evidence，就是证据。
在当前概率建模问题中，我们已经观察到了真实数据 x。
模型赋予这个观测数据的边缘概率或者概率密度，通常称为 Evidence。

把对数似然减去某个 KL 散度，表示成另外两个相对容易处理的项。
KL 散度本身有一个非常重要的性质： KL 散度永远大于或者等于零。

当两个分布在相关意义下完全一致时，KL 散度等于零。

Latent Variable Models

$$\underbrace{\mathbb{E}_{z\sim q(z)}\left[\log p_\theta(x|z)\right]}_{q_\phi(z|x)} \underbrace{- \mathcal{D}_{\mathrm{KL}}\left(q(z)||p_\theta(z)\right)}_{q_\phi(z|x)}$$

tractable tractable

- This is called Evidence Lower Bound (ELBO)
- Lower bound of $\log p_\theta(x)$
- This equation holds for any distribution $q(z)$
- Parameterize $q(z)$ by $q_\phi(z|x)$

区分两个参数符号。
θ 是生成模型的参数。
φ 是近似后验模型的参数。

qφ(z|x) 对应的网络是什么？
Encoder，也就是编码器

编码器的任务，是根据输入数据 x ，输出潜变量概率分布的参数。
我们并不是让编码器必须直接给出一个确定的 z，而是先得到一个概率分布，再从这个概率分布中采样。
这就是 VAE 和普通确定性自编码器的重要区别之一。
Encoder 负责从数据推断潜变量分布。

Decoder 负责根据潜变量生成数据分布。

Latent Variable Models

$$\underbrace{\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right]}_{\text{tractable}} \underbrace{-\mathcal{D}_{\mathrm{KL}}\left(q_\phi(z|x)||p(z)\right)}_{\text{tractable}}$$

- This is called Evidence Lower Bound (ELBO)
- Lower bound of $\log p_\theta(x)$

- This equation holds for any distribution $q(z)$

p(z)选择一个简单、已知的潜变量先验分布，计算两个概率分布之间的 KL 散度。

- Parameterize $q(z)$ by $q_\phi(z|x)$
- let $p_\theta(z)$ be a simple known prior $p(z)$

如果 pθ(z) 本身也是一个非常复杂的神经网络概率模型，那么这个 KL 散度可能仍然不好计算。

为什么有了这个下界，就可以用来训练生成模型？我们举一个例子。

假设有一张真实图片 x。
当前模型给这张图片的对数似然是 -10。但是，我们无法直接计算这个 -10。
不过，通过刚才的 ELBO 表达式，我们可能能够计算出一个下界，比如 -15。
现在，我们调整神经网络参数，让这个下界从 -15 提高到 -12。
这代表什么？代表我们当前优化的这个下界变大了。从训练目标的角度来看，这可能有助于提高原来的对数似然。

对于同一个模型，ELBO 的数值提高，并不严格保证真实对数似然一定同步提高。

因为 ELBO 与真实对数似然之间，还存在一个后验 KL 散度。

这两者的差距也可能发生变化。

所以更加准确的理解是：

我们通过最大化一个可以处理的下界，来间接优化生成模型的概率学习目标。
这和直接最大化真实对数似然并不完全相同，但在高维潜变量模型中更加具有可操作性。

Variational Autoencoder

Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$

我们会在这个简单的高斯分布上进行采样，然后把采样得到的 z 输入一个复杂的神经网络 Decoder。

由于神经网络具有较强的非线性表达能力，所以即使输入分布比较简单，也可以通过复杂映射生成具有丰富结构的数据。

我们可以想象：

左边是一个简单的二维高斯分布。

中间是一个包含许多神经网络层的 Decoder。

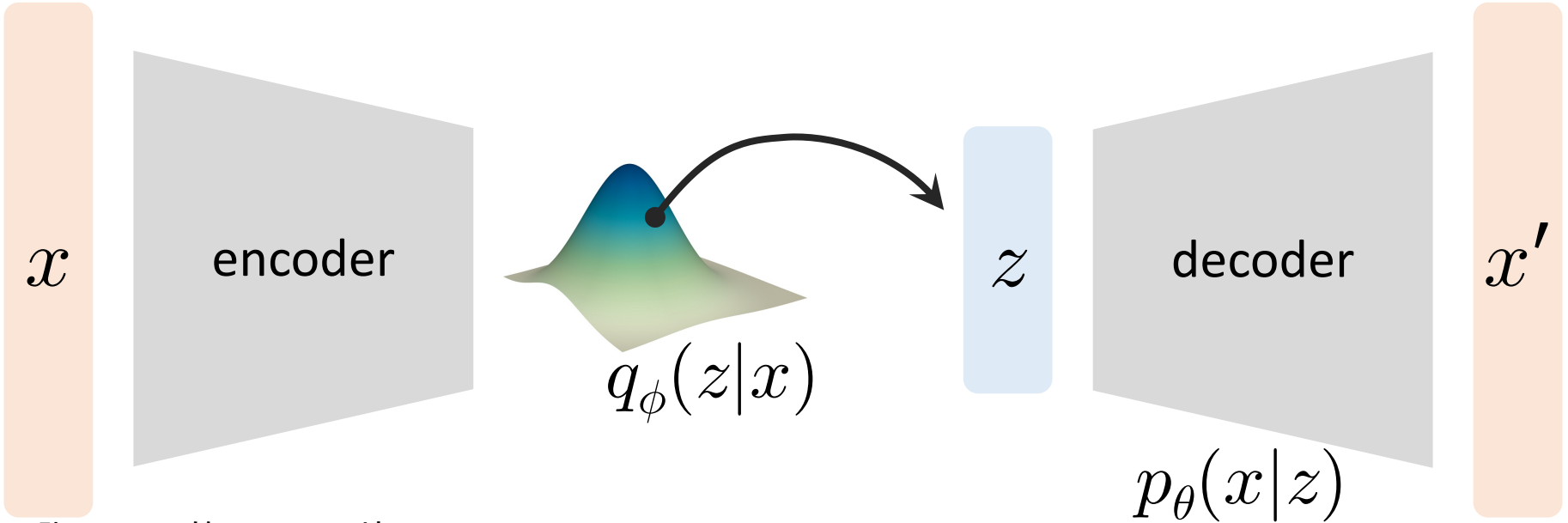
右边则是各种不同的图片。

神经网络要学习的，就是如何将这种简单的潜在概率结构映射成复杂的图像分布。

Variational Autoencoder

Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$



1. 假设我们输入一张 64×64 的 RGB 图片
2. 64×64×3 = 12288个像素通道数值
3. Encoder 可以是卷积神经网络，也可以是其他适合处理图片的网络结构。
4. **VAE 并不是简单地把 Encoder 的输出直接交给 Decoder，而是会根据编码器所描述的概率分布进行采样**
5. Decoder 利用这个 z，预测原始图片对应的生成分布。
6. Decoder 可以把这个 16 维向量，转换成一个包含 12288 个数值的输出，用来表示图片像素分布

Variational Autoencoder

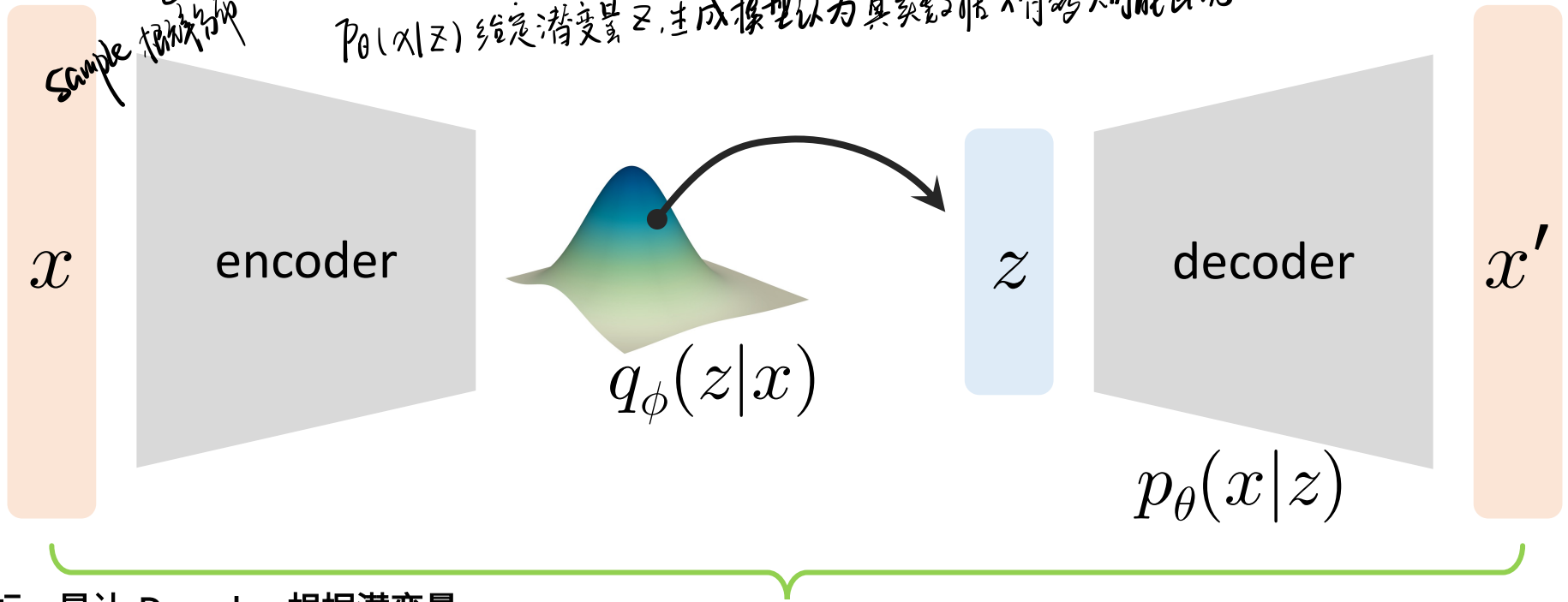
Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$

max

Sample 抽样得到

$p_\theta(x|z)$ 给定潜变量 z，生成模型认为真实的数据 x 有多大可能出现



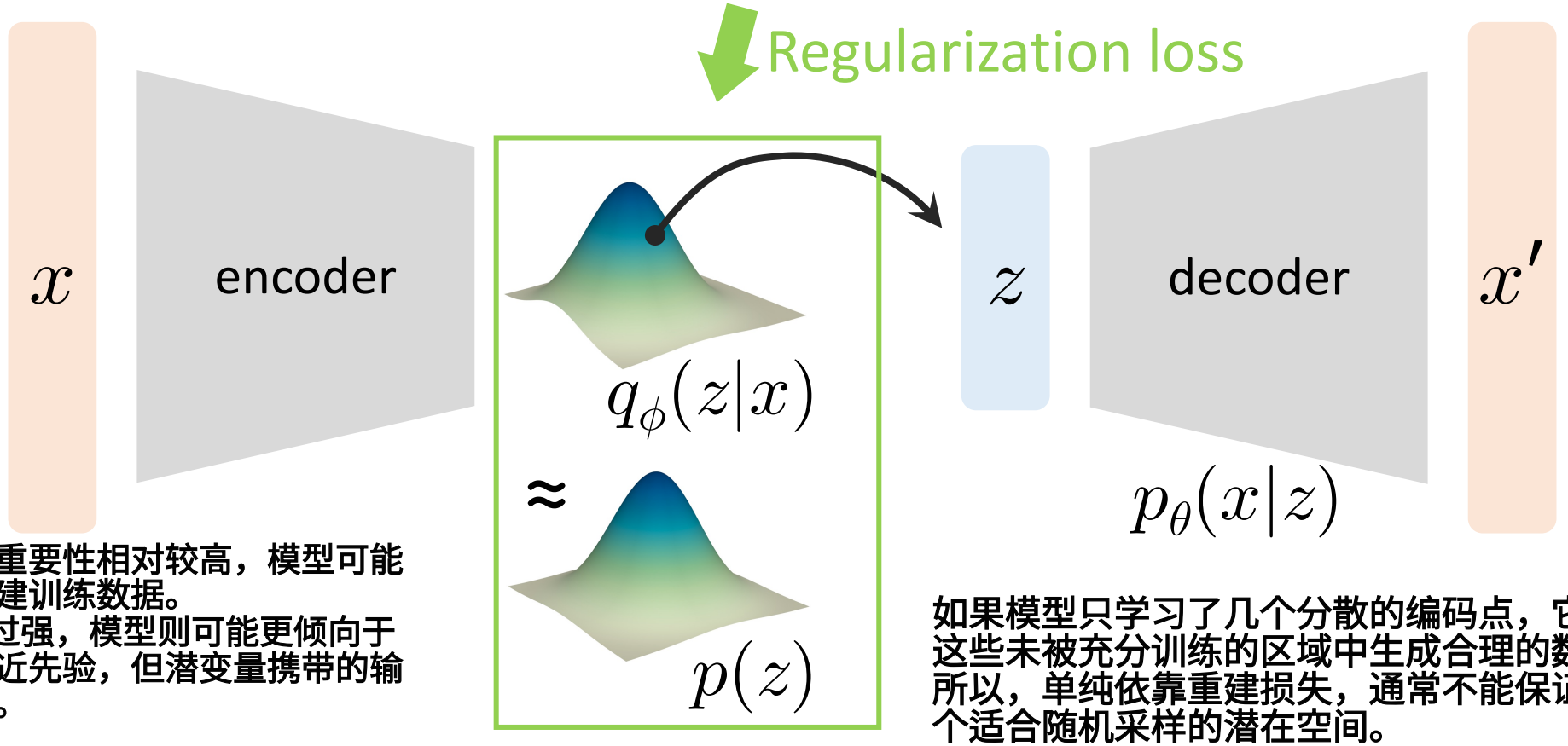
重建损失的根本目标，是让 Decoder 根据潜变量产生的条件概率分布，能够很好地解释真实输入数据。

Variational Autoencoder

Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$

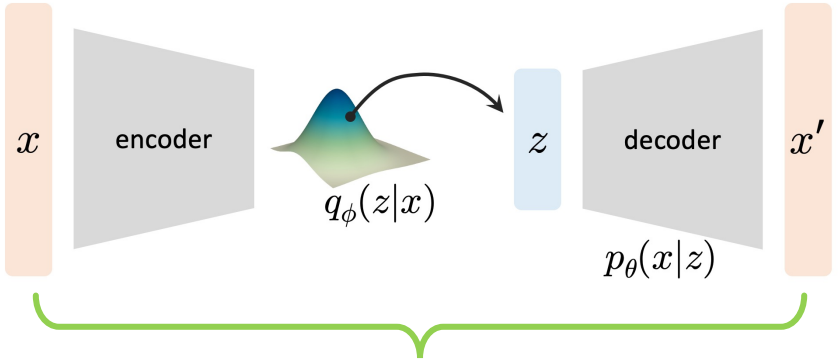
Regularization Loss，也就是正则化损失



如果重建损失的重要性相对较高，模型可能更倾向于精确重建训练数据。
如果 KL 正则化过强，模型则可能更倾向于让编码器分布接近先验，但潜变量携带的输入信息可能减少。

如果模型只学习了几个分散的编码点，它可能无法在这些未被充分训练的区域中生成合理的数据。
所以，单纯依靠重建损失，通常不能保证我们获得一个适合随机采样的潜在空间。

Variational Autoencoder



Reconstruction loss

$$\mathcal{L}_{\theta,\phi}(x) = \boxed{-\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big]} + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$

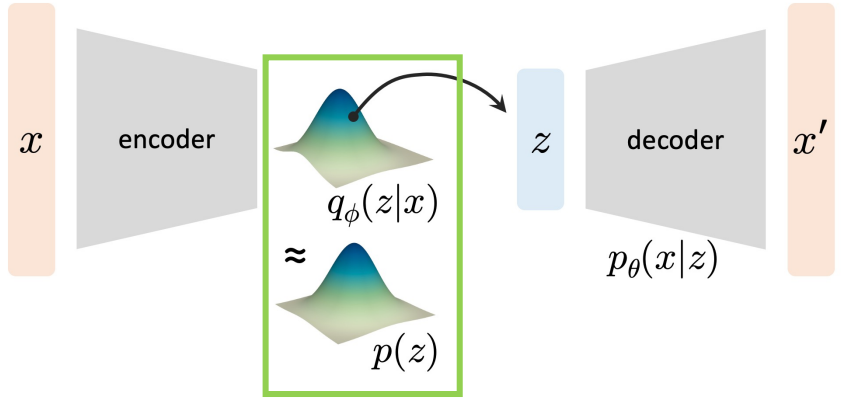
Example: L2 loss

- one-step Monte Carlo: $z \sim q_\phi(z|x)$
- map z by decoder net: $\boxed{g_\theta(z) \to x'}$ network estimates distribution's parameters
- model $p_\theta(x|z)$ by Gaussian: $p_\theta(x|z) = \mathcal{N}(x \mid x', \sigma_0^2)$ (assume fixed std)
- negative log likelihood: $\frac{1}{2\sigma_0^2}||x - x'||^2 + const$

当我们假设观测数据在给定潜变量后服从固定方差的高斯分布时：**最大化高斯条件似然，就等价于最小化平方 L2 重建误差。**

- L2 loss ⇒ a Gaussian neighborhood around data point x

Variational Autoencoder



$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right] + \mathcal{D}_{\mathrm{KL}}\left(q_\phi(z|x)||p(z)\right)$$

Regularization loss

VAE 的两项损失都具体化了
第一项，重建损失。
在固定方差高斯观测模型下，可以对应平方 L2 Loss。
第二项，KL 正则化。
在高斯近似后验与标准正态先验的设定下，可以通过解析公式计算。
因此，VAE 的损失函数现在已经具有了可以在程序中实现的具体形式。

Example: Gaussian prior

- let $p(z) = \mathcal{N}(z \mid 0, \mathbf{I})$
- model $q_\phi(z|x)$ by Gaussian: $\mathcal{N}(z \mid \mu, \sigma)$
- map x by encoder net: $f_\phi(x) \to \mu, \sigma$ again, network estimates distribution's parameters
- compute loss analytically: $\mathcal{D}_{\mathrm{KL}}\left(\mathcal{N}(z \mid \mu, \sigma) \,||\, \mathcal{N}(z \mid 0, \mathbf{I})\right)$ (see pset 1)

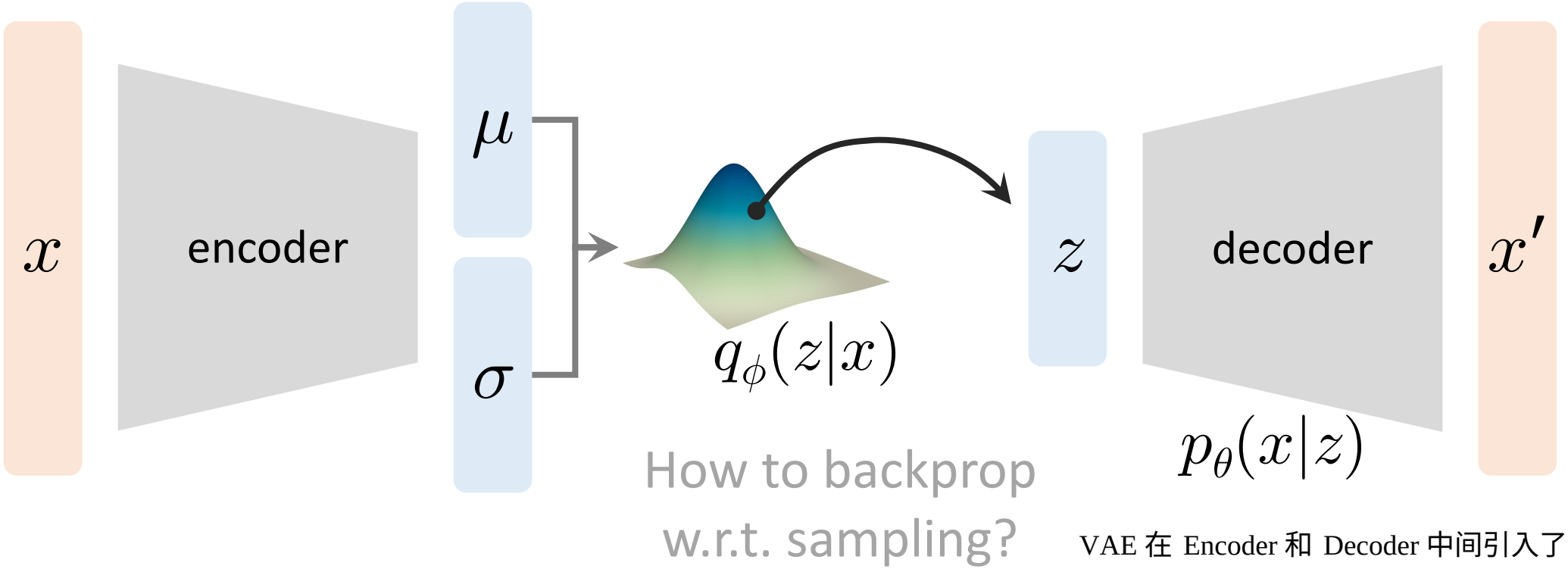
- fixed covariance ⇒ L2 loss on μ (see pset 1)

Variational Autoencoder 重参数化技巧

Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right] + \mathcal{D}_{\mathrm{KL}}\left(q_\phi(z|x)||p(z)\right)$$

在普通的前馈神经网络中，每一层通常都是由某种可微分的数学运算构成。
例如：矩阵乘法。加法。激活函数。
这些运算中的绝大多数，都可以计算梯度。



我们应该如何让重建损失的梯度，经过随机采样这一步，传回 Encoder？

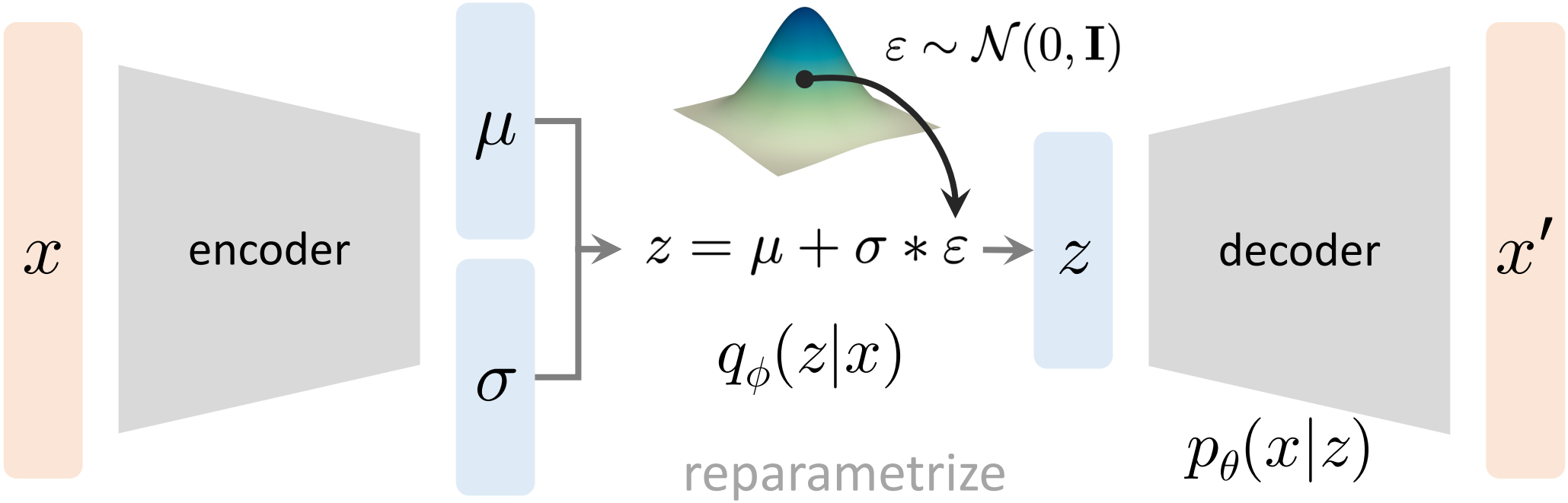
VAE 在 Encoder 和 Decoder 中间引入了一次随机采样。
我们只是把这个随机采样步骤当成一个不透明的黑箱，那么普通自动求导系统就无法直接沿着这一步得到一条明确的、针对μ 和 σ 的可微计算路径。

Variational Autoencoder

这不代表包含随机变量的概率模型在数学上完全无法求梯度。

Maximize ELBO ⇒ minimize an objective:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$



首先，μ 是编码器预测出来的均值。
σ 是编码器预测出来的标准差。
ε 是一个从标准正态分布中采样得到的随机变量。

我们是直接从：
$q_\varphi(z|x)$
中采样得到 z。
而在这一页，课件把采样过程重新写成了一个显式公式：
$$z = \mu + \sigma \times \varepsilon$$
其中：
$$\varepsilon \sim N(0,I)$$
这就是著名的重参数化技巧。

现在，我们不再直接从$N(\mu,\sigma^2)$ 里面采样 z。
而是先从最简单的标准正态分布中采样一个ε。
然后，对ε 进行平移和缩放。
平移的大小由μ 决定。
缩放的大小由σ 决定。
最终得到：
$z = \mu + \sigma\varepsilon$

在一次具体的前向计算中，我们可以把已经采样得到的ε，看成这次计算图中的一个固定随机输入。
于是：
$z = \mu + \sigma\varepsilon$
就成为一个关于μ 和 σ 的可微函数。

我们把依赖模型参数的随机采样，改写成固定分布的随机噪声与可微确定性变换的组合，使重建损失能够通过采样路径对编码器参数进行梯度优化。

Variational Autoencoder

... so far, we have discussed an objective on one x:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$$

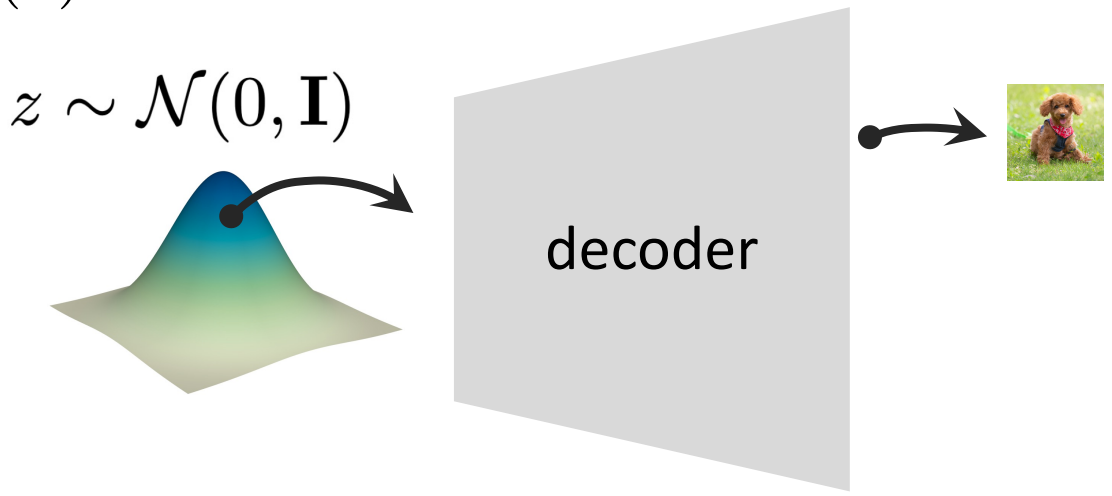
Overall loss is expectation over data:

$$\mathcal{L}_{\theta,\phi} = \mathbb{E}_{x\sim p_{data}(x)}\Big[-\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + \mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)\Big]$$

Variational Autoencoder

Inference (generation):

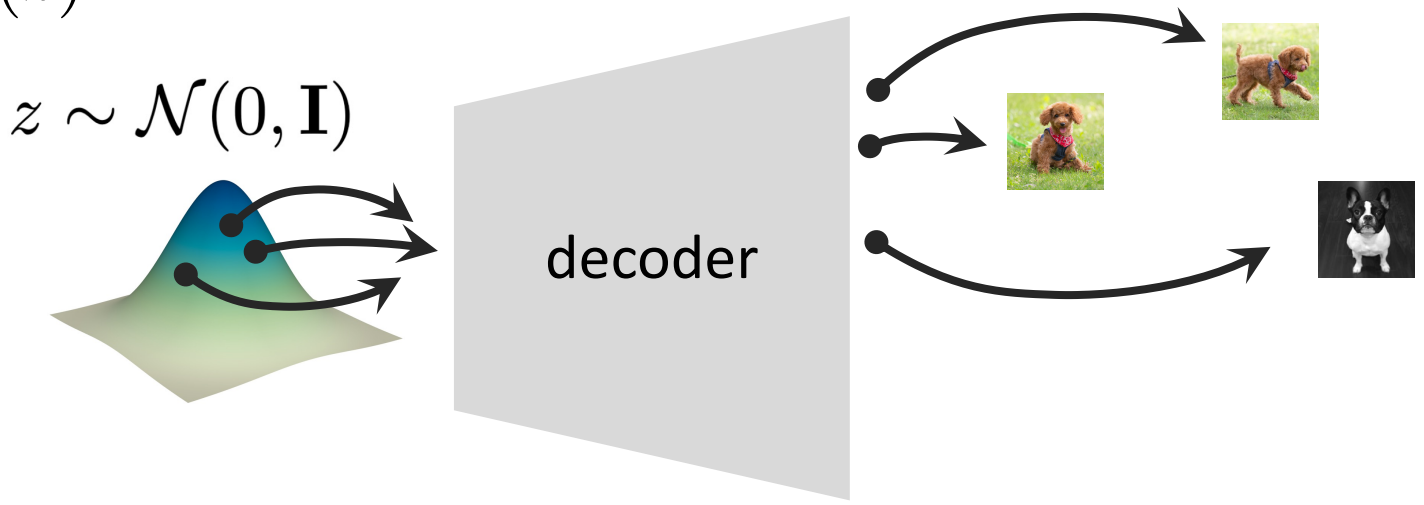
- sample z from: $\mathcal{N}(0, \mathbf{I})$
- map z by decoder net: $g_\theta(z)$



Variational Autoencoder

Inference (generation):

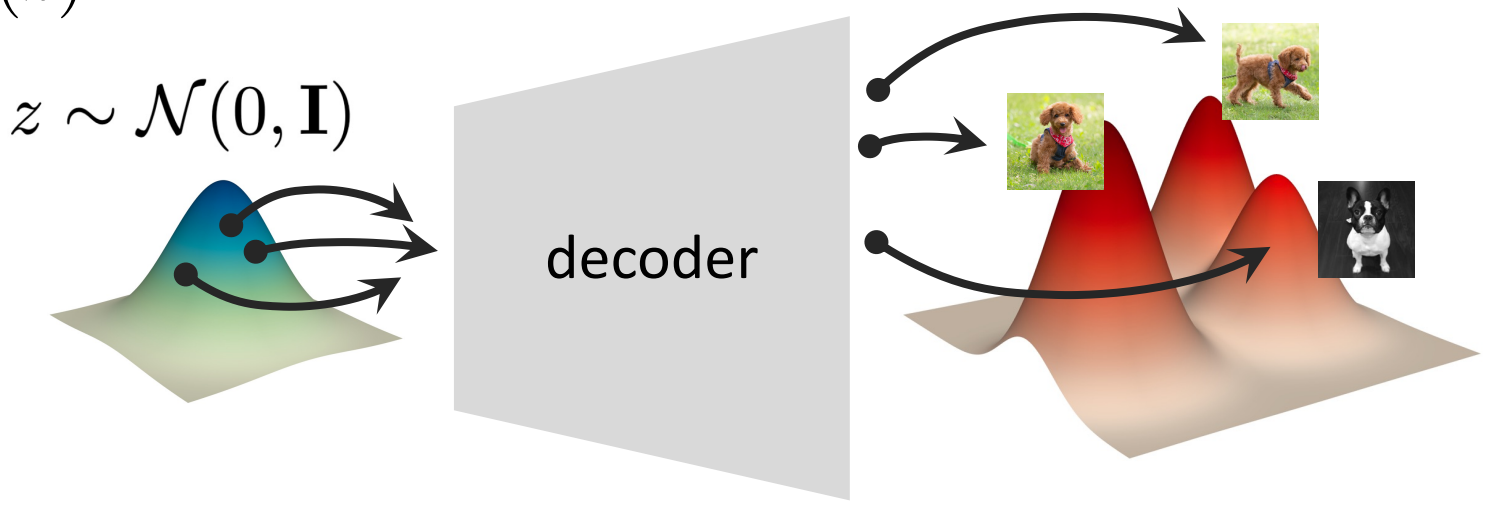
- sample z from: $\mathcal{N}(0, \mathbf{I})$
- map z by decoder net: $g_\theta(z)$



Variational Autoencoder

Inference (generation):

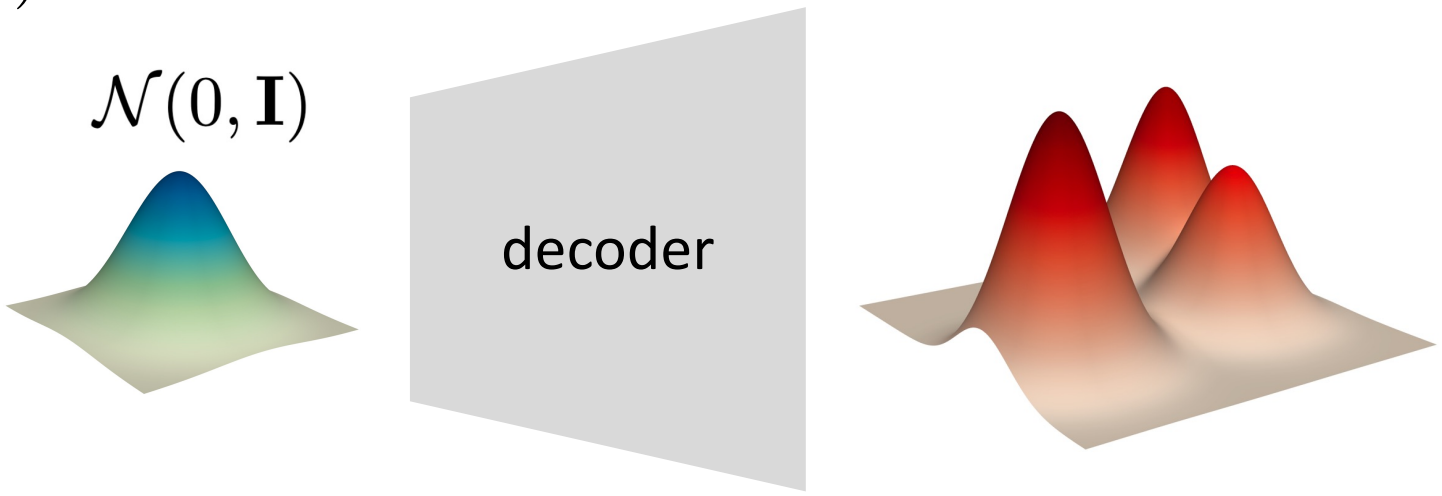
- sample z from: $\mathcal{N}(0, \mathbf{I})$
- map z by decoder net: $g_\theta(z)$



Variational Autoencoder

Inference (generation):

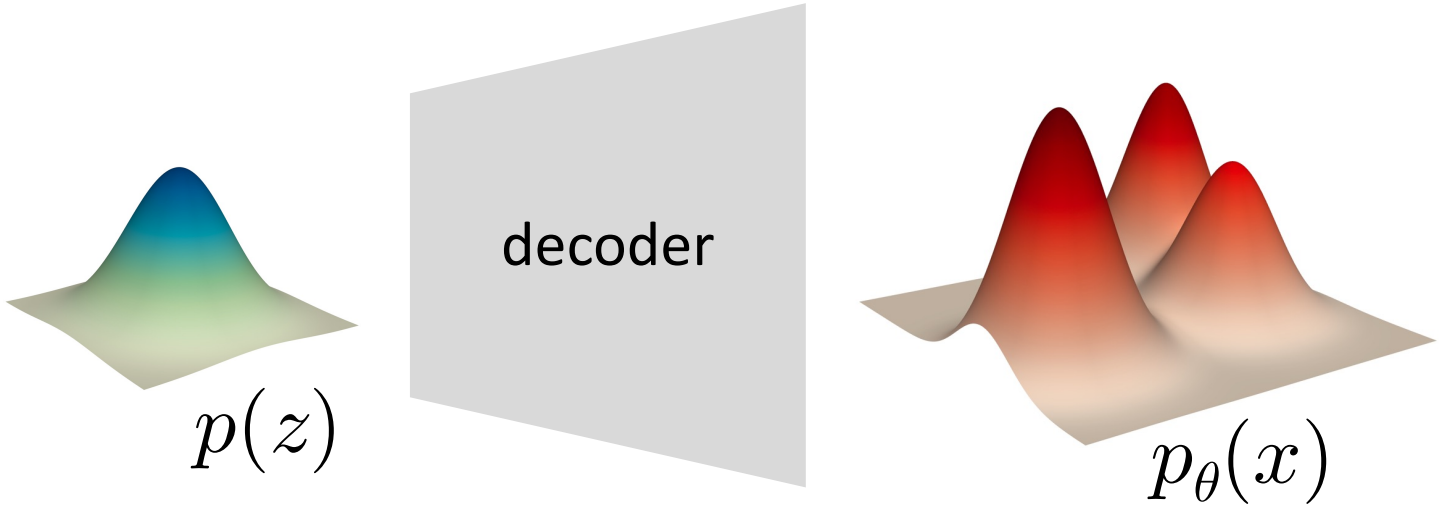
- sample z from: $\mathcal{N}(0, \mathbf{I})$
- map z by decoder net: $g_\theta(z)$



Decoder is a deterministic mapping from one distribution to another.

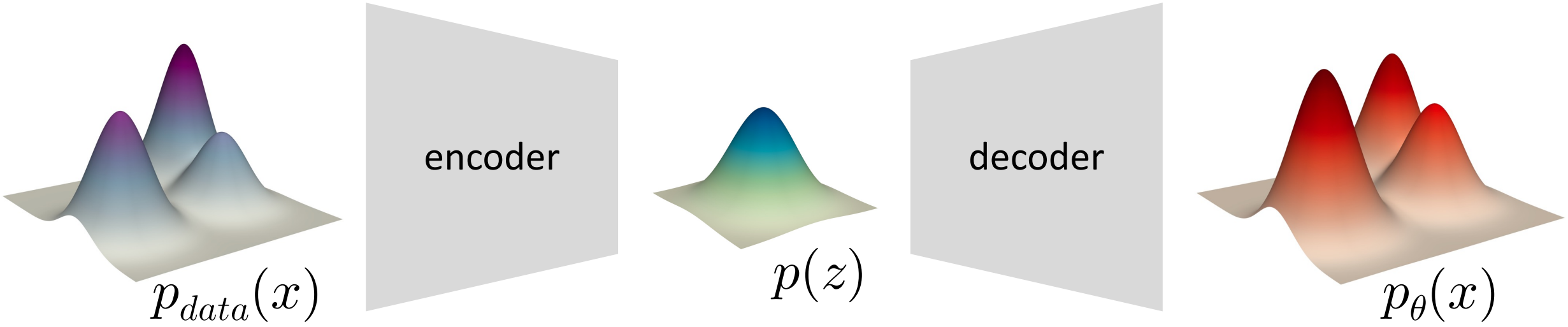
A view of “Autoencoding Distributions”

- decoder: maps latent distribution to data distribution



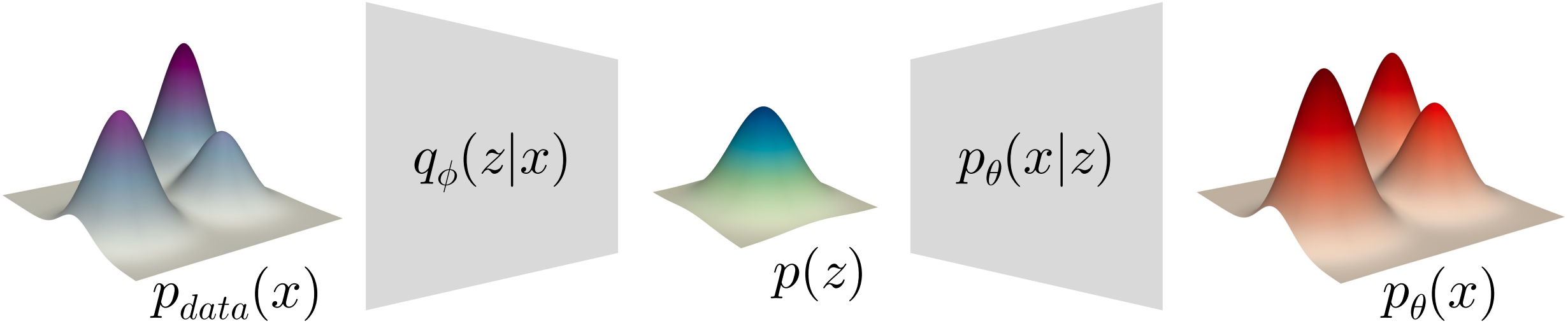
A view of “Autoencoding Distributions”

- encoder: maps data distribution to latent distribution
- decoder: maps latent distribution to data distribution



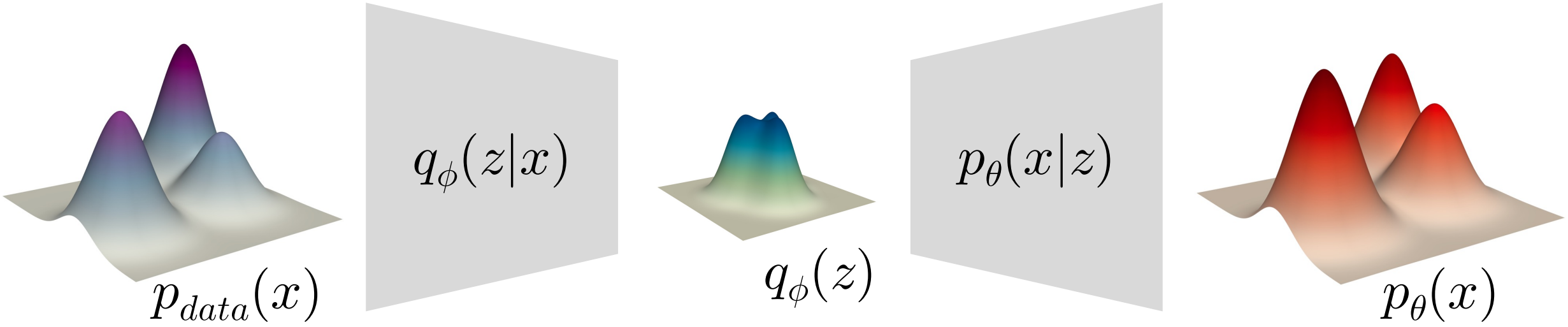
A view of “Autoencoding Distributions”

- encoder: maps data distribution to latent distribution
- decoder: maps latent distribution to data distribution



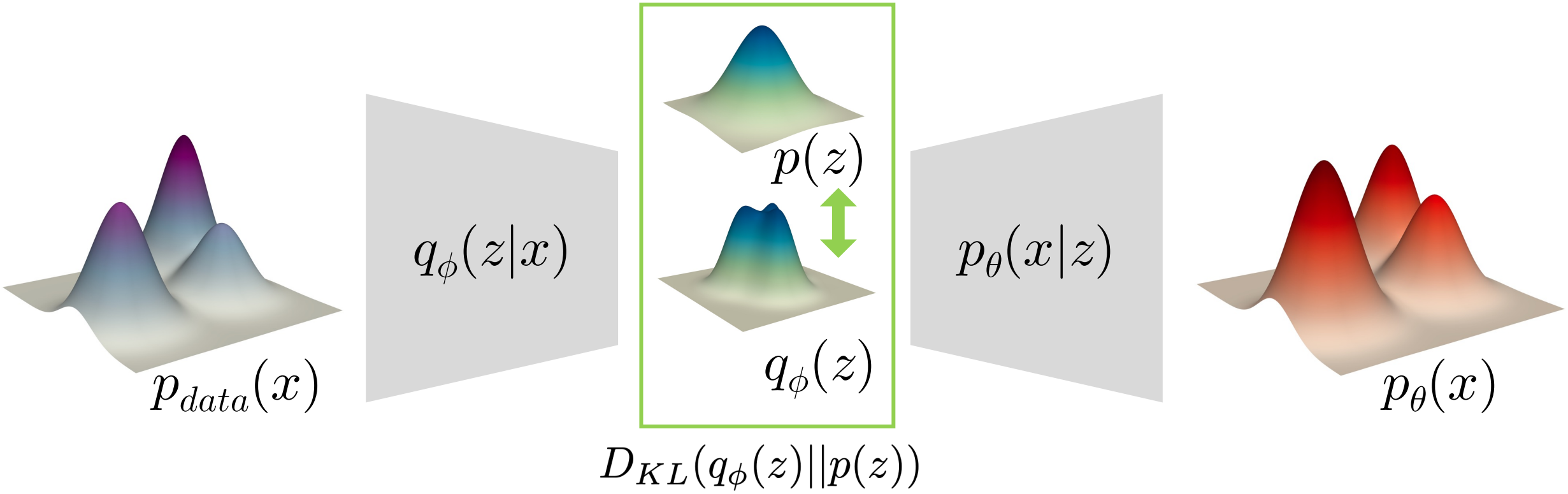
A view of “Autoencoding Distributions”

- encoded latent distribution: $q_\phi(z) = \int_x q_\phi(z|x) p_{data}(x) dx$



A view of “Autoencoding Distributions”

- encoded latent distribution: $q_\phi(z) = \int_x q_\phi(z|x) p_{data}(x) dx$



E.g., see “*InfoVAE: Information Maximizing Variational Autoencoders*”, 2017

Illustration

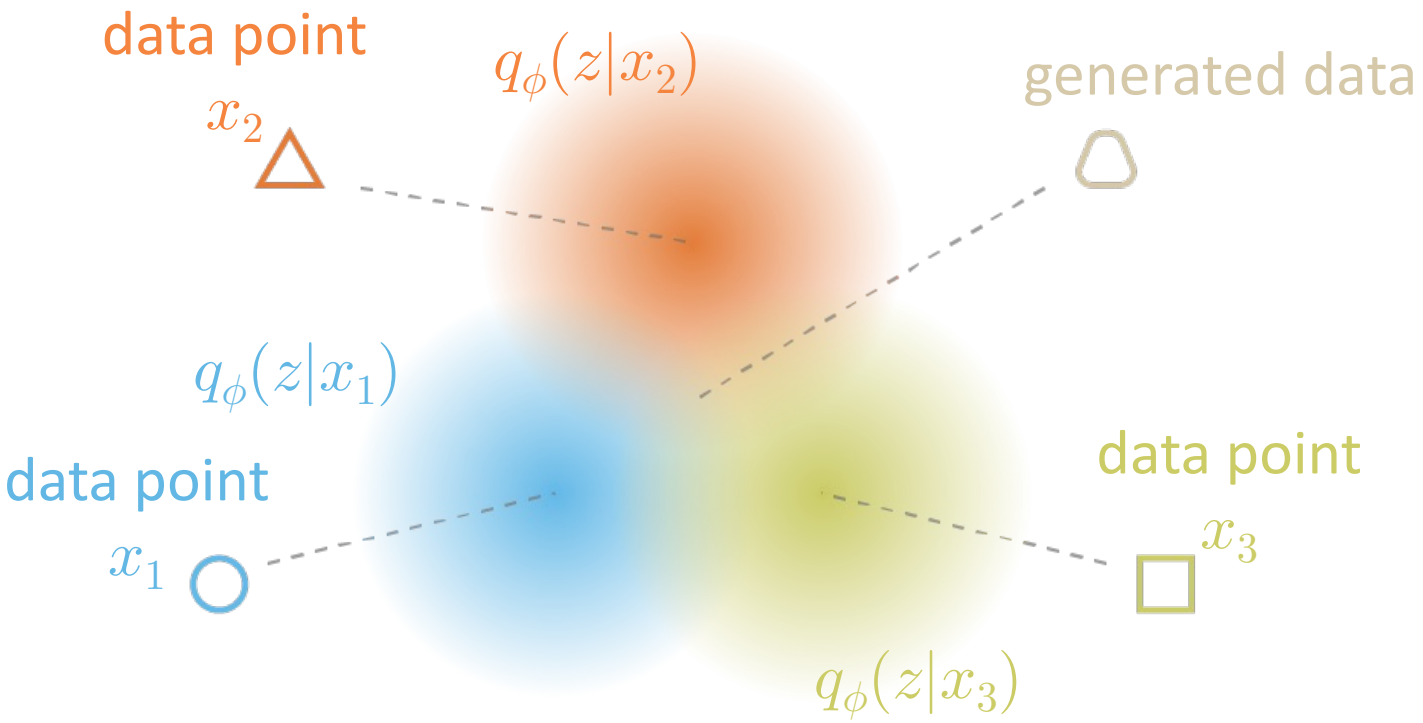
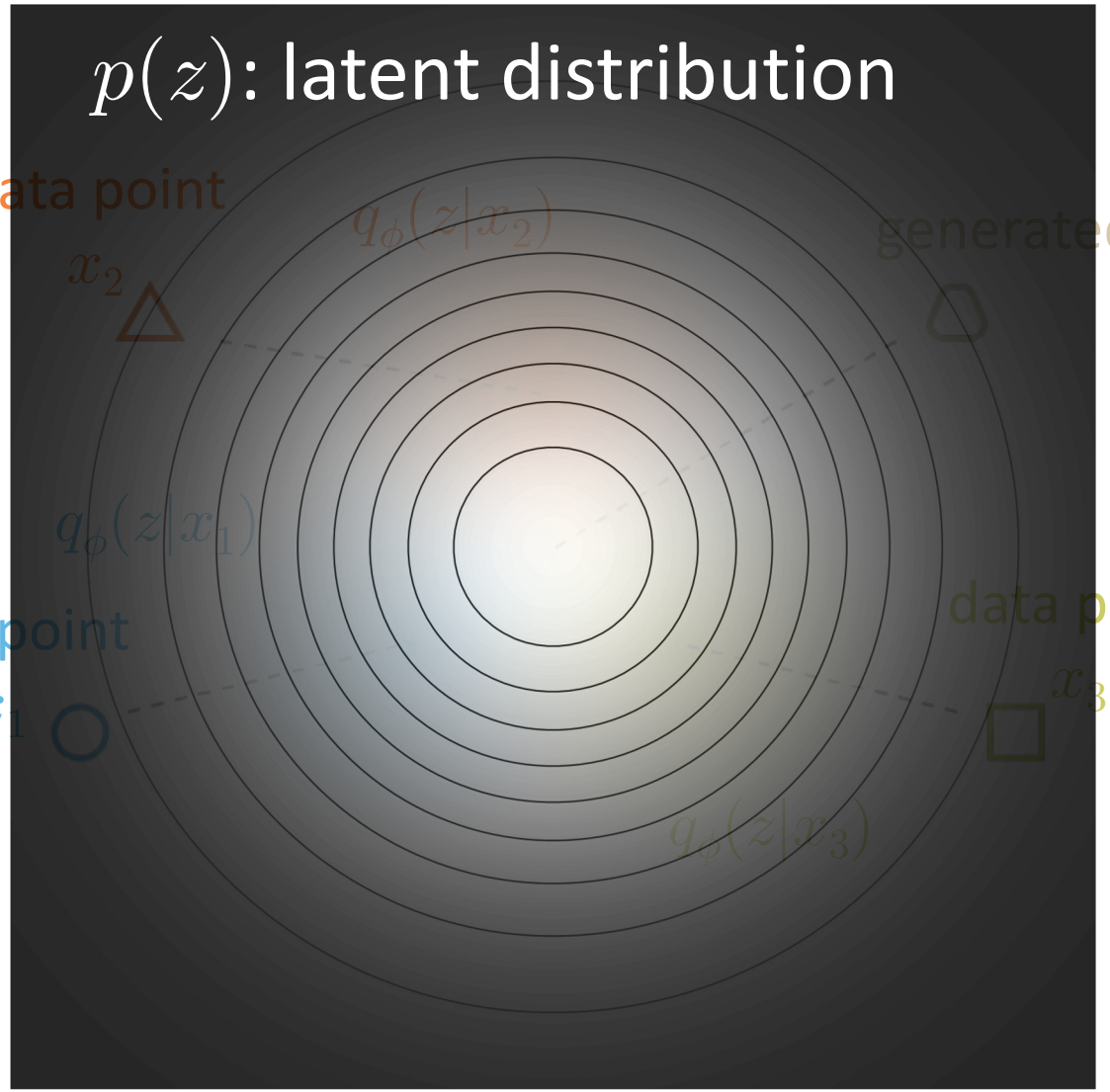


Figure adapted from: Joseph Rocca “Understanding Variational Autoencoders (VAEs)”
https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73

Illustration



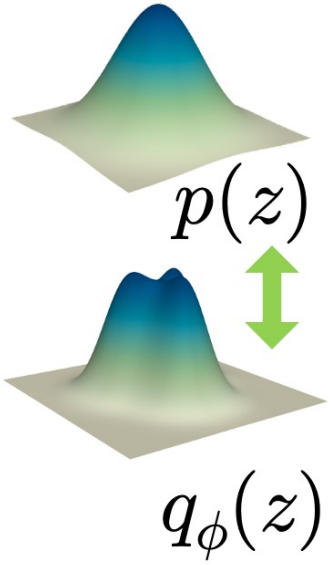
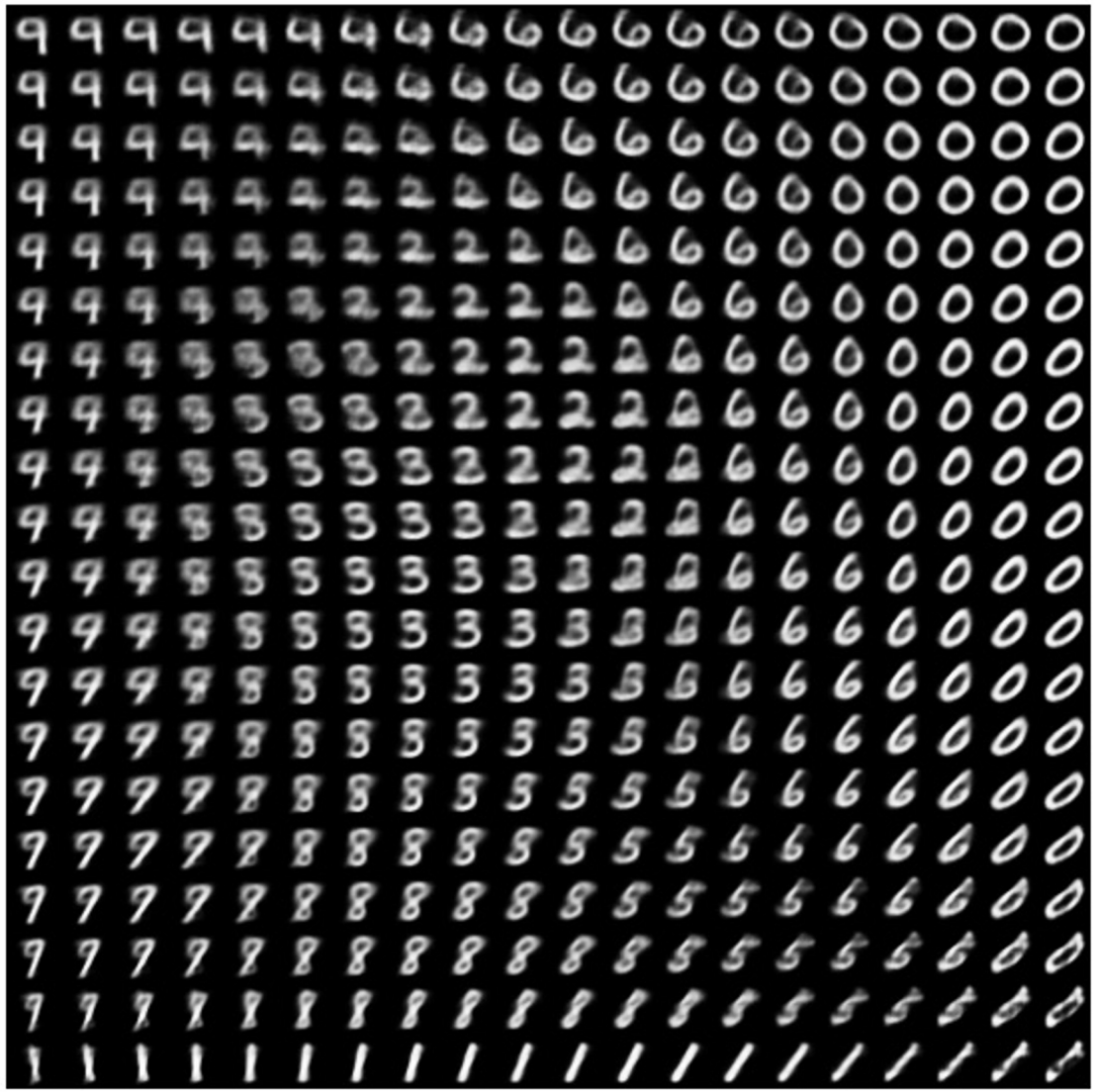


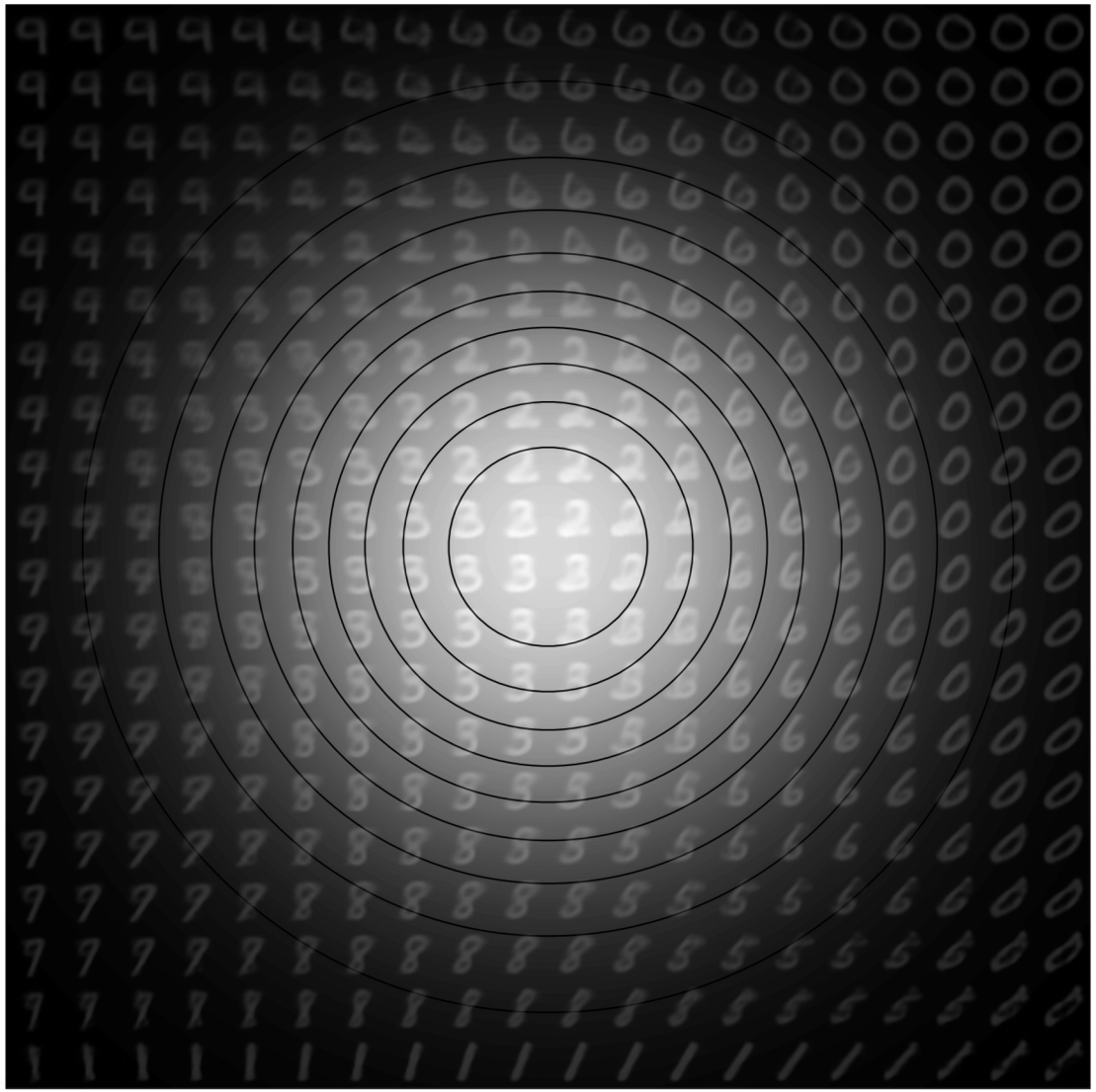
Figure adapted from: Joseph Rocca “Understanding Variational Autoencoders (VAEs)”
https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73

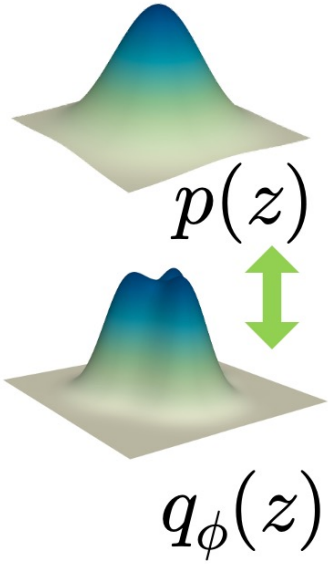
VAE: 2D latent space on MNIST



"Convolutional Variational Autoencoder"
https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/generative/cvae.ipynb

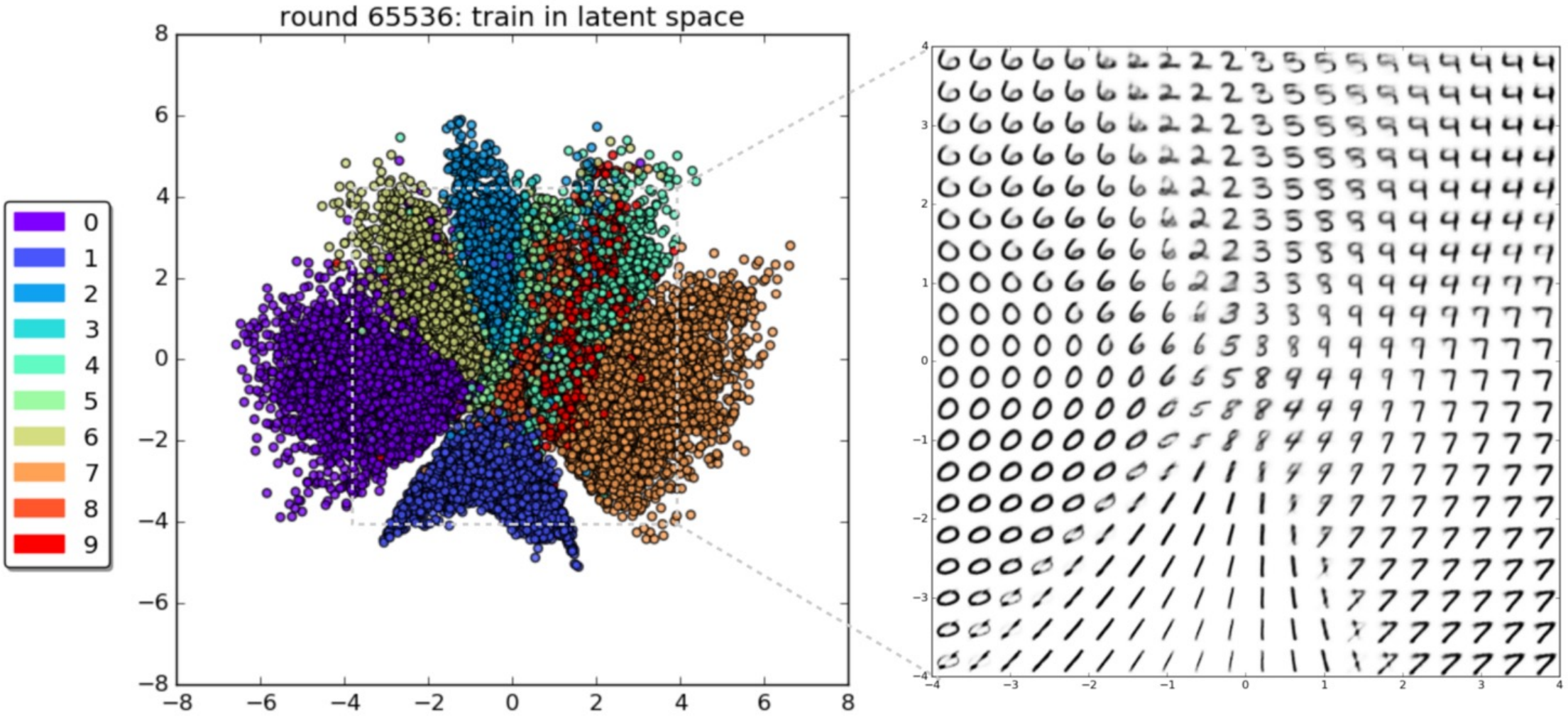
VAE: 2D latent space on MNIST





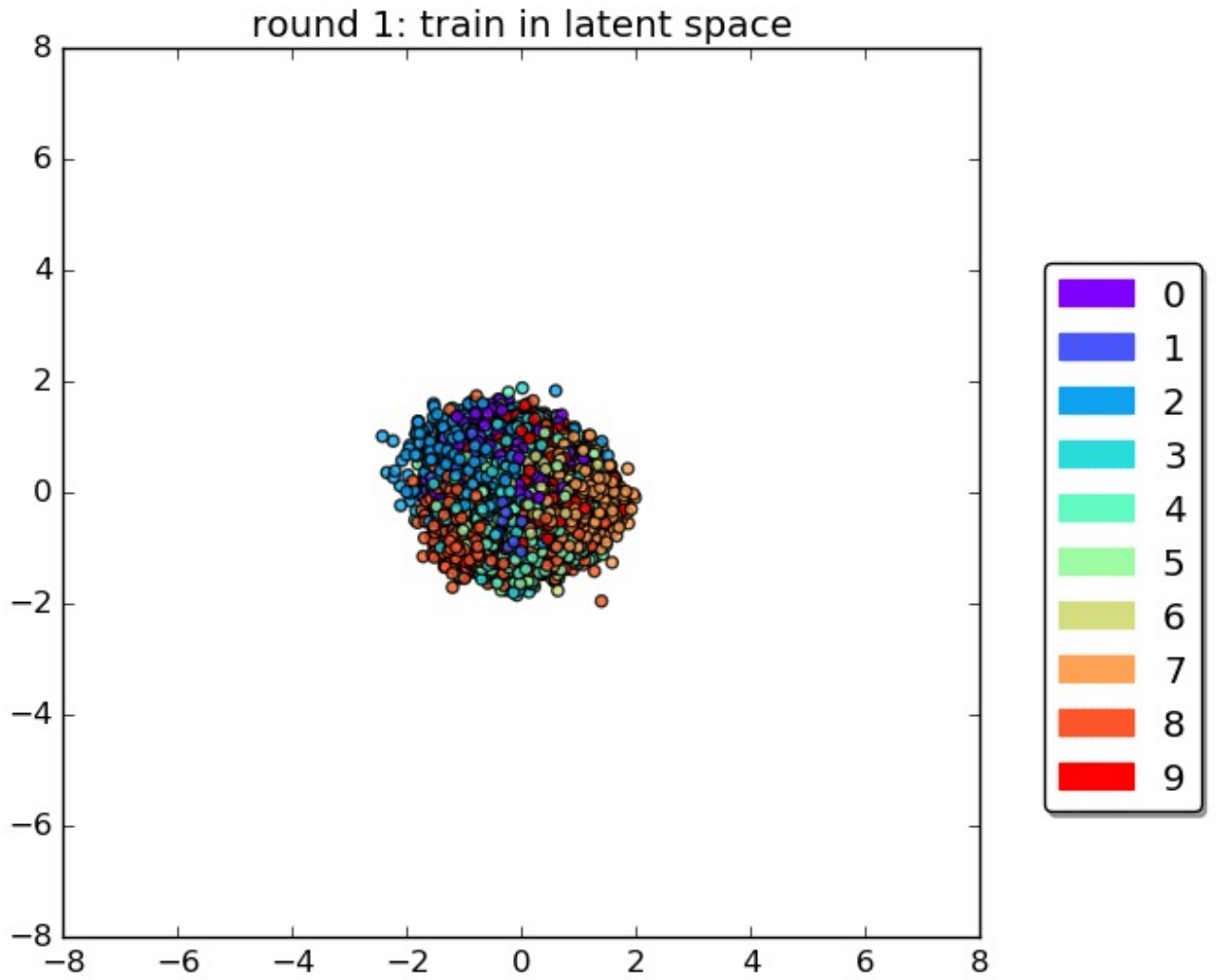
"Convolutional Variational Autoencoder"
https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/generative/cvae.ipynb

VAE: 2D latent space on MNIST



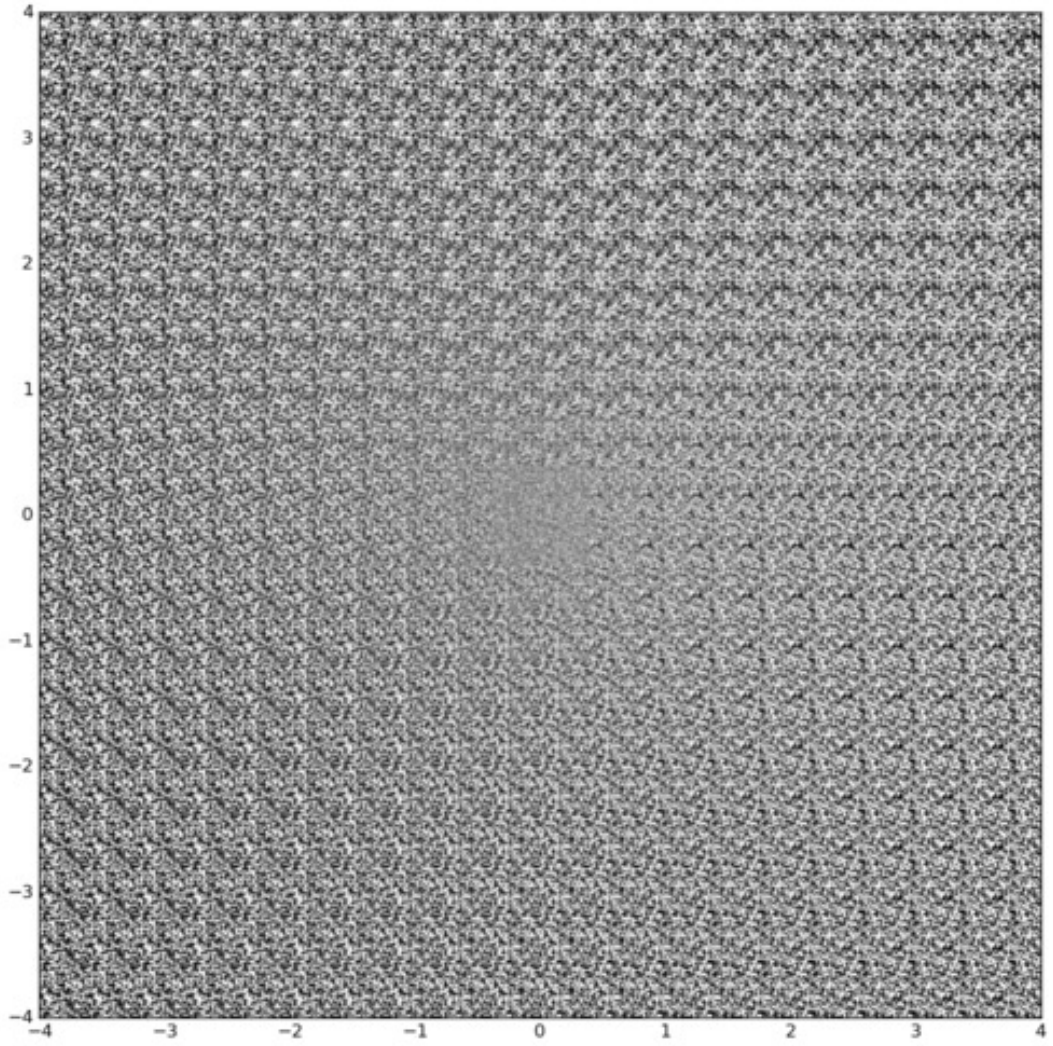
"Introducing Variational Autoencoders (in Prose and Code)"
https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html

VAE: 2D latent space on MNIST



To pdf users: this is animation. Check it on: "Introducing Variational Autoencoders (in Prose and Code)"
https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html

VAE: 2D latent space on MNIST



To pdf users: this is animation. Check it on: "Introducing Variational Autoencoders (in Prose and Code)"
https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html

VAE: 2D latent space on “Frey Face” dataset



Kingma and Welling. Auto-Encoding Variational Bayes, ICLR 2014

Relation to Expectation-Maximization (EM)

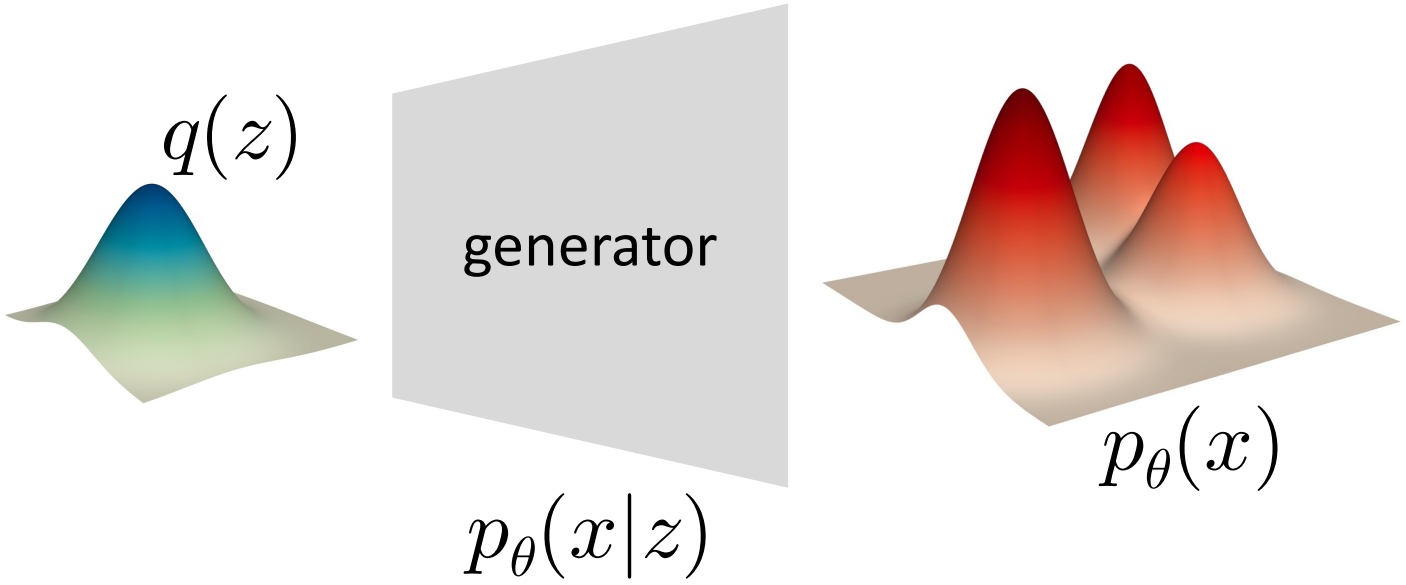
Recap: Latent Variable Models

Two sets of variables:

- q: distribution of latent
- θ: parameters of generator

VAE:

- parametrize q by a network
- stochastic gradient decent

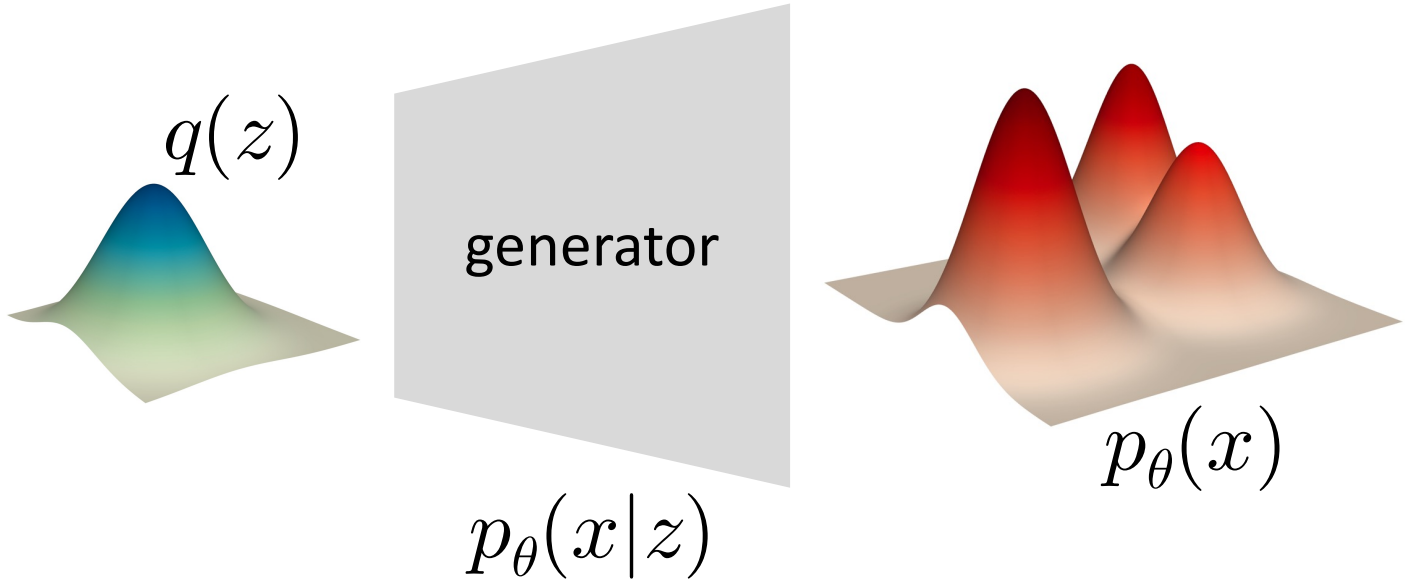


Expectation-Maximization (EM):

- often parametrize q analytically
- coordinate descent (i.e., alternating optimization)

EM as A Max-Max Procedure

$$\text{ELBO} = \mathbb{E}_{x \sim p_{data}(x)} \Big[\mathbb{E}_{z \sim q(z|x)} \Big[\log p_\theta(x|z) \Big] - \mathcal{D}_{\text{KL}} \Big(q(z|x) || p(z) \Big) \Big]$$



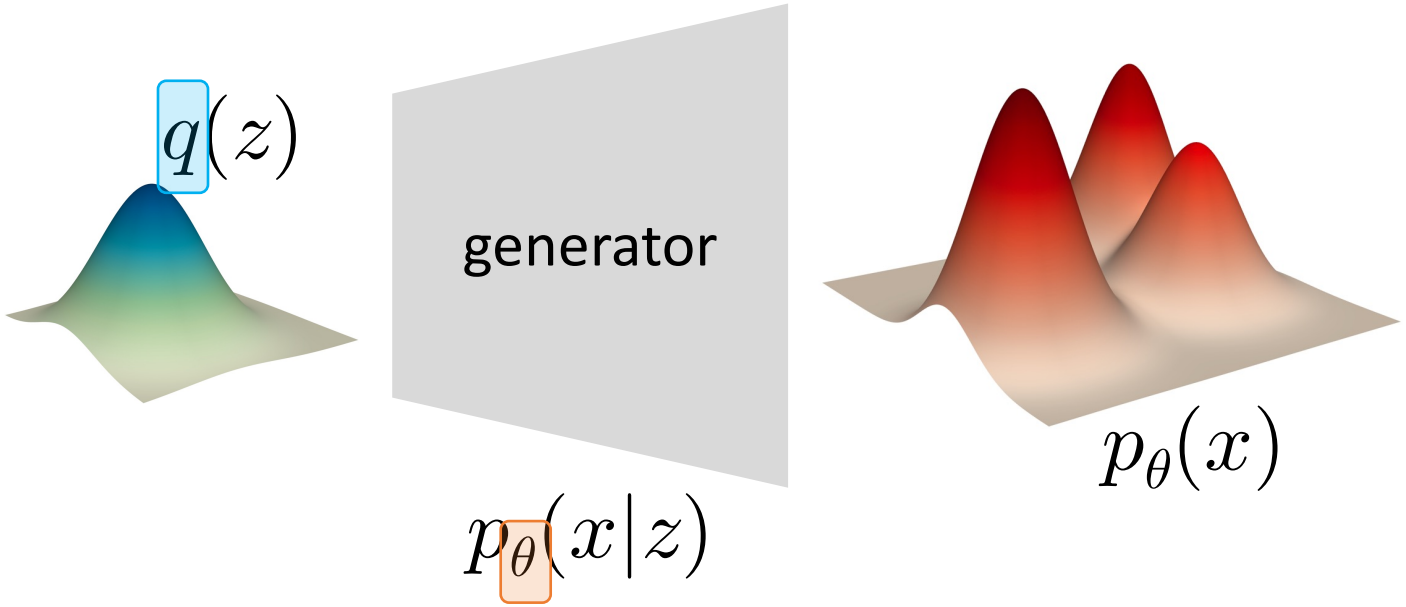
EM as A Max-Max Procedure

$$\text{ELBO} = \mathbb{E}_{x \sim p_{data}(x)} \Big[\mathbb{E}_{z \sim q(z|x)} \Big[\log p_\theta(x|z) \Big] - \mathcal{D}_{\text{KL}} \Big(q(z|x) || p(z) \Big) \Big]$$

$$\max_{\theta, q} \text{ELBO}(\theta, q(\cdot))$$

Two sets of variables:

- q - distribution of latent
- θ - parameters of generator



Coordinate descent:

- max-max procedure (GAN: max-min)

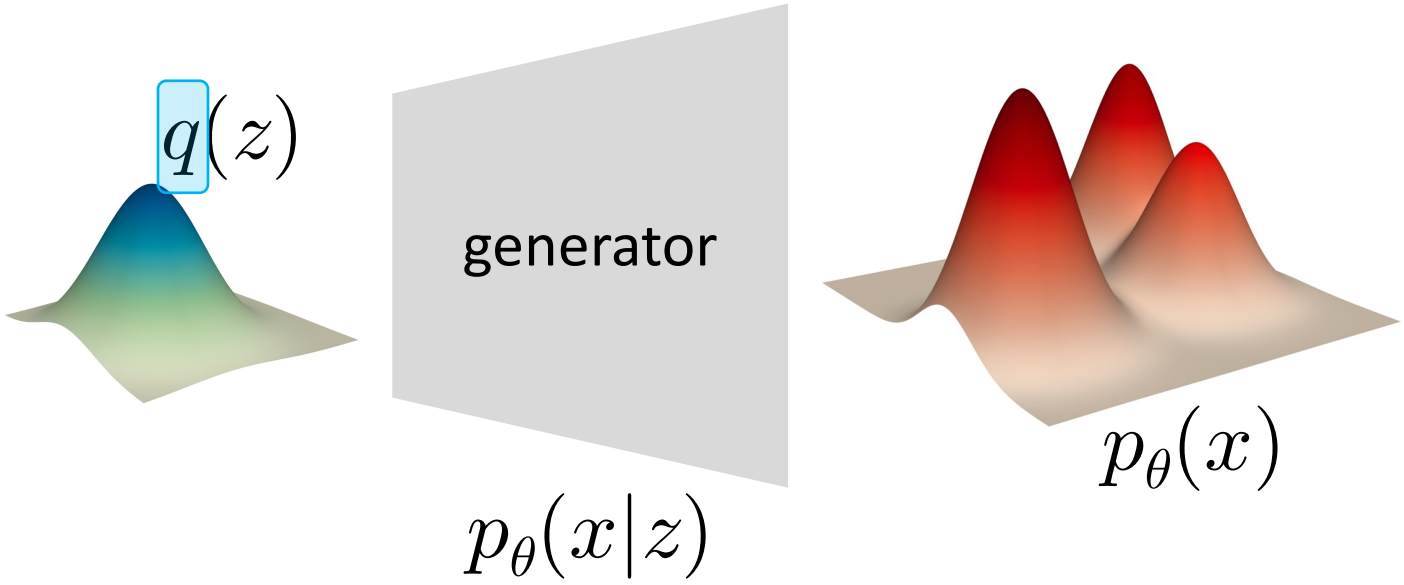
EM as A Max-Max Procedure

$$\text{ELBO} = \mathbb{E}_{x\sim p_{data}(x)}\Big[\mathbb{E}_{z\sim q(z|x)}\Big[\log p_\theta(x|z)\Big] - \mathcal{D}_{\text{KL}}\Big(q(z|x)||p(z)\Big)\Big]$$

$$\max_{\theta, q} \text{ELBO}\big(\theta, q(\cdot)\big)$$

E-step: optimize for q

$$q^{(t)} = p_{\theta^{(t)}}(z|x)$$



Details can be found in: Hastie et al. “The Elements of Statistical Learning”.

EM as A Max-Max Procedure

$$\text{ELBO} = \mathbb{E}_{x \sim p_{data}(x)} \Big[\mathbb{E}_{z \sim q(z|x)} \Big[\log p_\theta(x|z) \Big] - \mathcal{D}_{\text{KL}} \Big(q(z|x) || p(z) \Big) \Big]$$

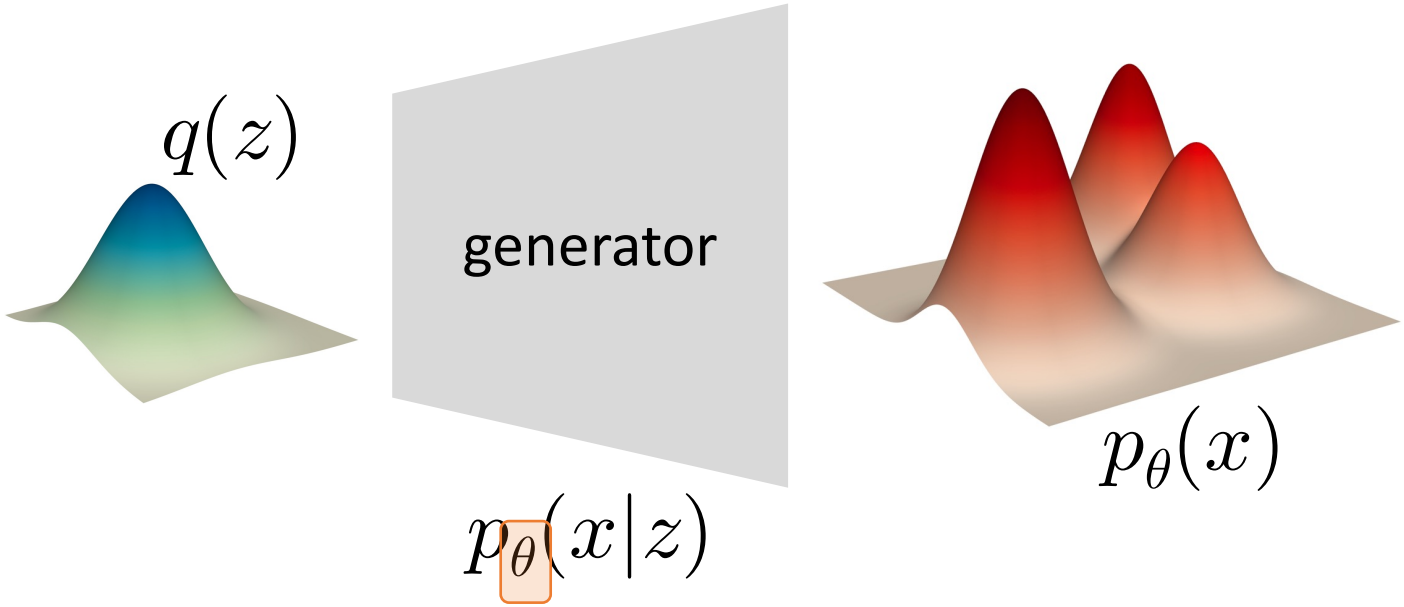
$$\max_{\theta, q} \text{ELBO}\big(\theta, q(\cdot)\big)$$

E-step: optimize for q

$$q^{(t)} = p_{\theta^{(t)}}(z|x)$$

M-step: optimize for θ

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta|\theta^{(t)})$$



with sub-objective defined as: $Q(\theta|\theta^{(t)}) = \mathbb{E}_{p_{data}(x)} \mathbb{E}_{p_{\theta^{(t)}}(z|x)} [\log p_\theta(x, z)]$

Details can be found in: Hastie et al. “The Elements of Statistical Learning”.

EM as A Max-Max Procedure

$$\text{ELBO} = \mathbb{E}_{x \sim p_{data}(x)} \left[\mathbb{E}_{z \sim q(z|x)} \left[\log p_\theta \right. \right.$$

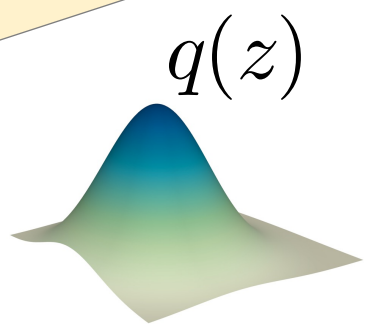
$$\max_{\theta, q} \text{ELBO}(\theta, q(\cdot))$$

q: often in analytical forms

- Gaussian Mixtures
- **K-means**

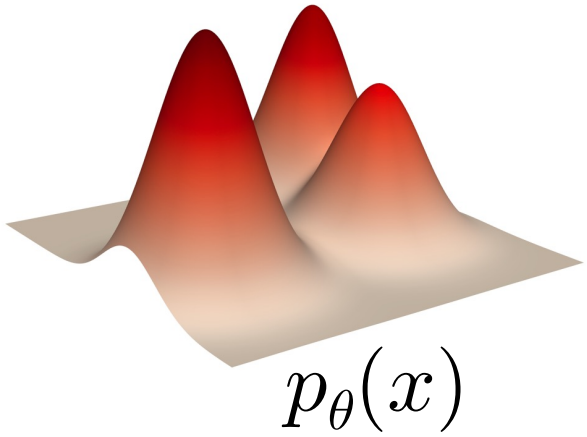
E-step: optimize for q

$$q^{(t)} = p_{\theta^{(t)}}(z|x)$$



generator

$p_\theta(x|z)$



M-step: optimize for θ

$$\theta^{(t+1)} = \arg\max_\theta Q(\theta|\theta^{(t)})$$

with sub-objective defined as: $Q(\theta|\theta^{(t)}) = \mathbb{E}_{p_{data}(x)} \mathbb{E}_{p_{\theta^{(t)}}(z|x)} [\log p_\theta(x, z)]$

Details can be found in: Hastie et al. “The Elements of Statistical Learning”.

A running example of EM: K-means

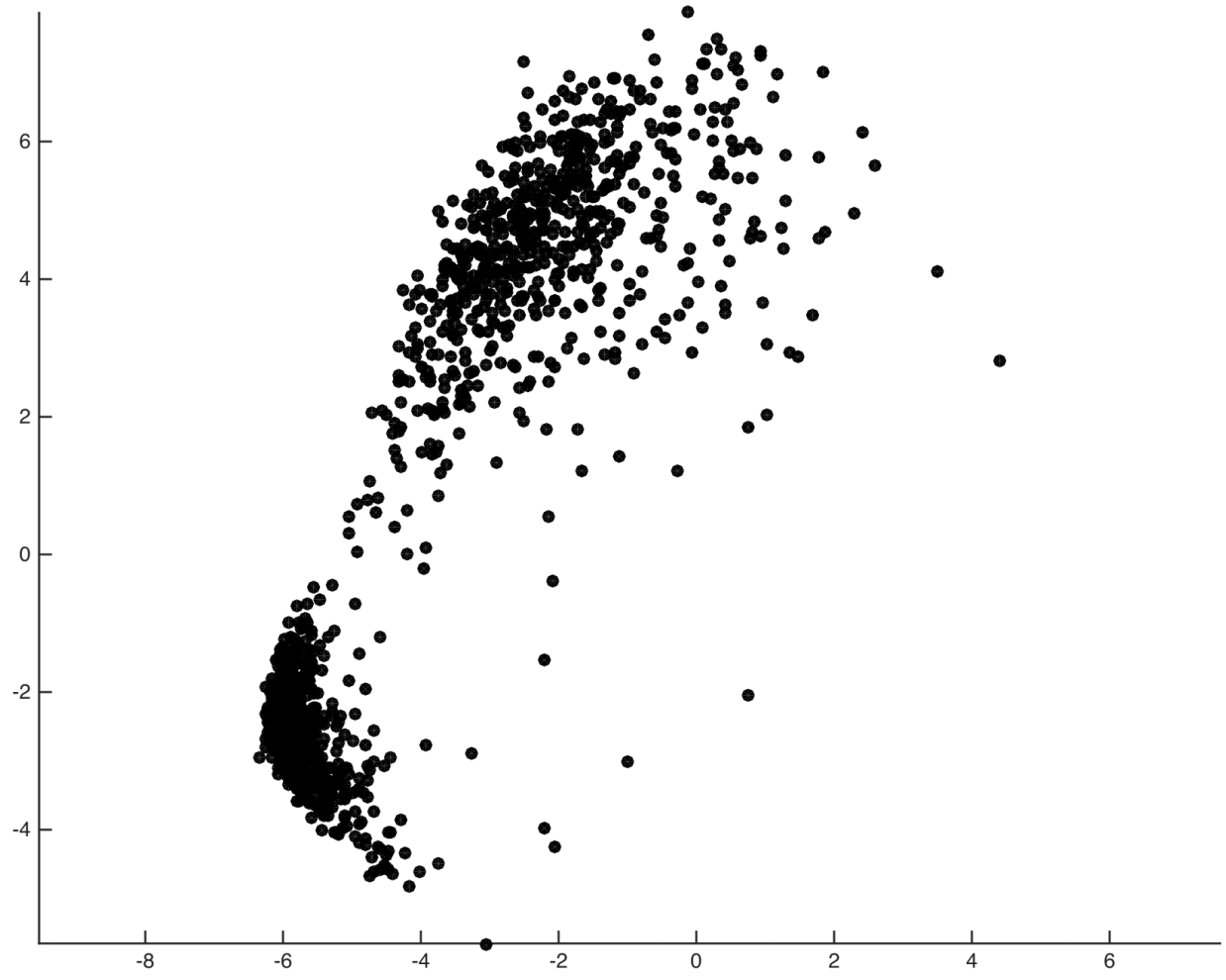


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ

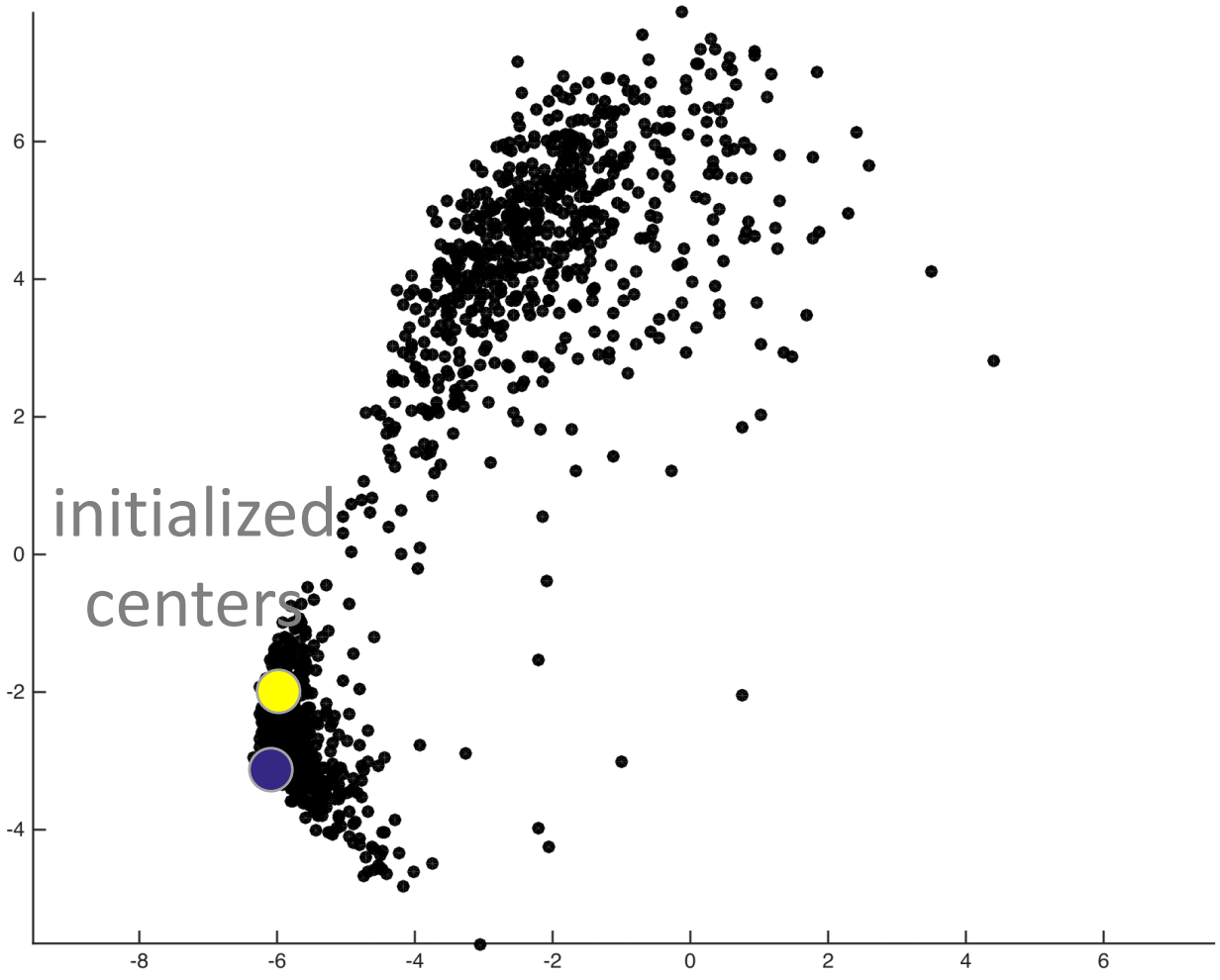


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ
- assignment:

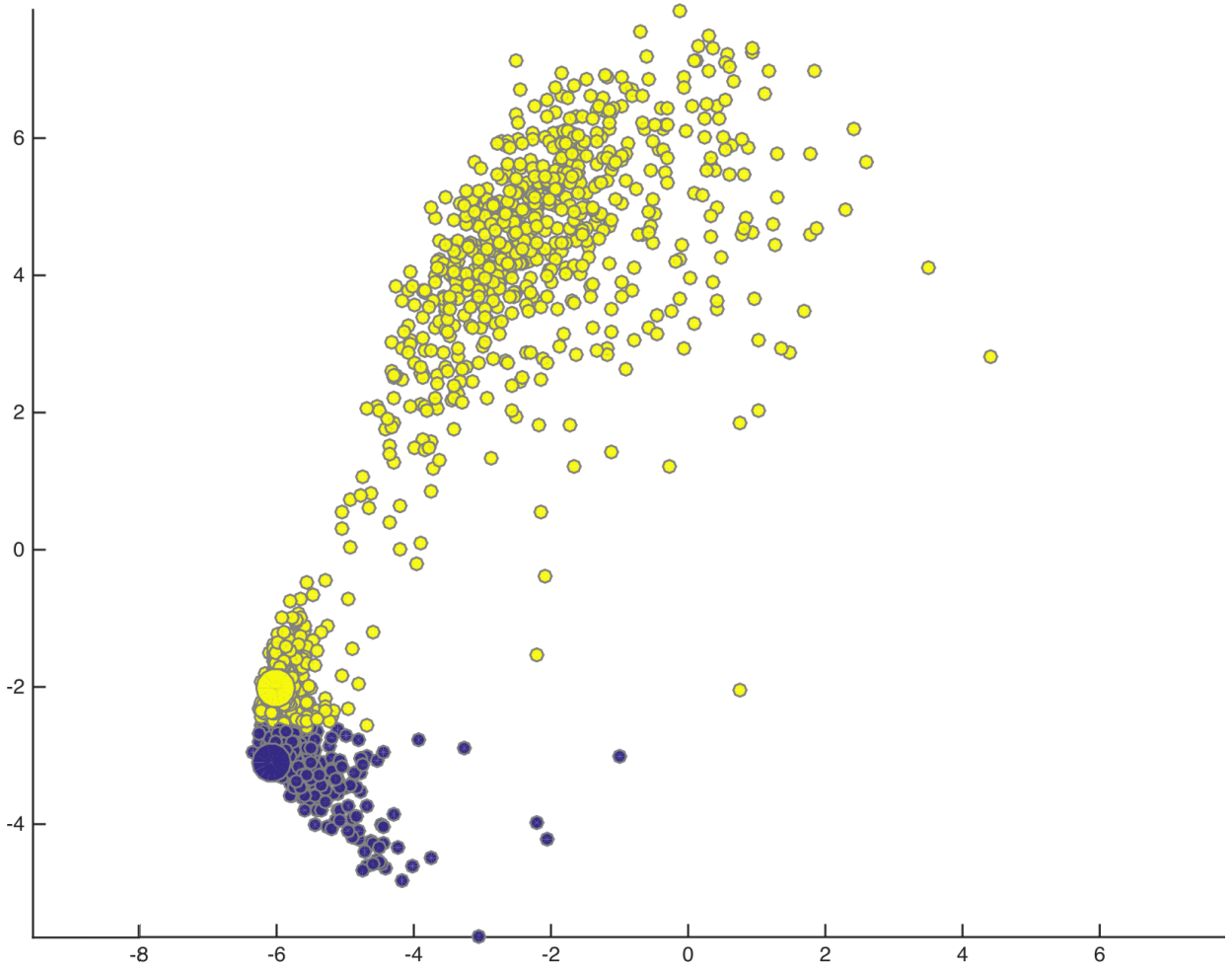


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ
- assignment: E-step

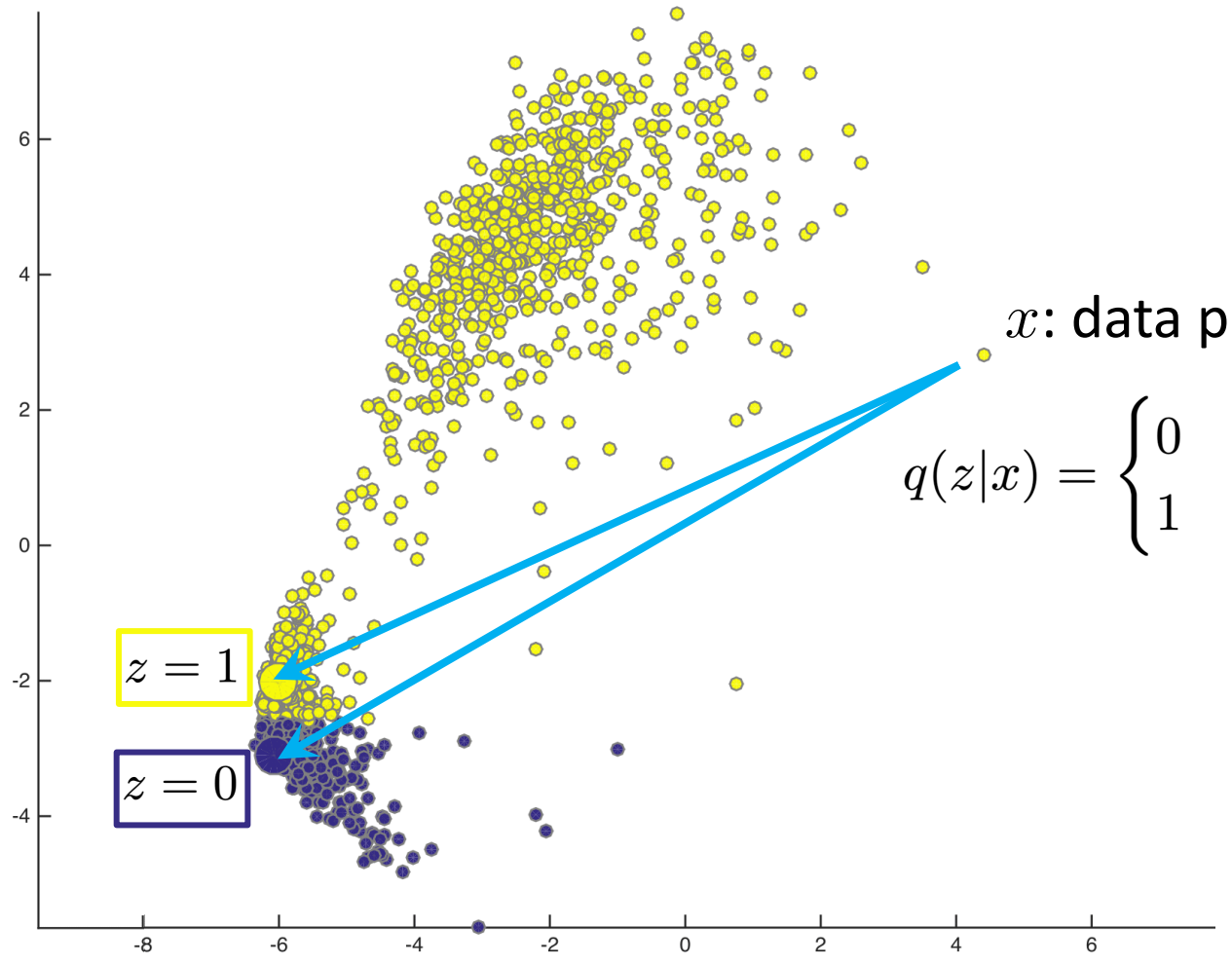


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ
- assignment: E-step

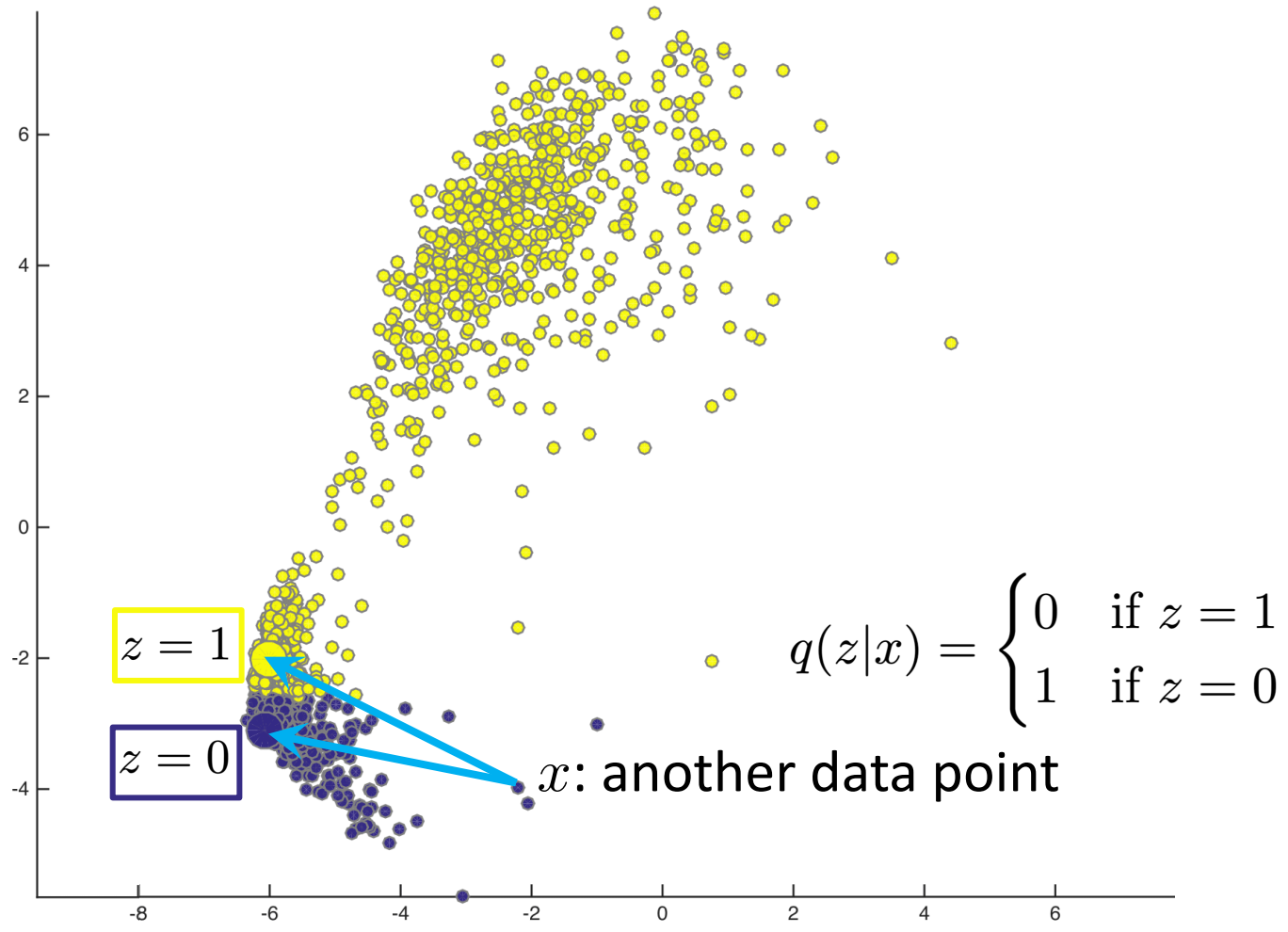
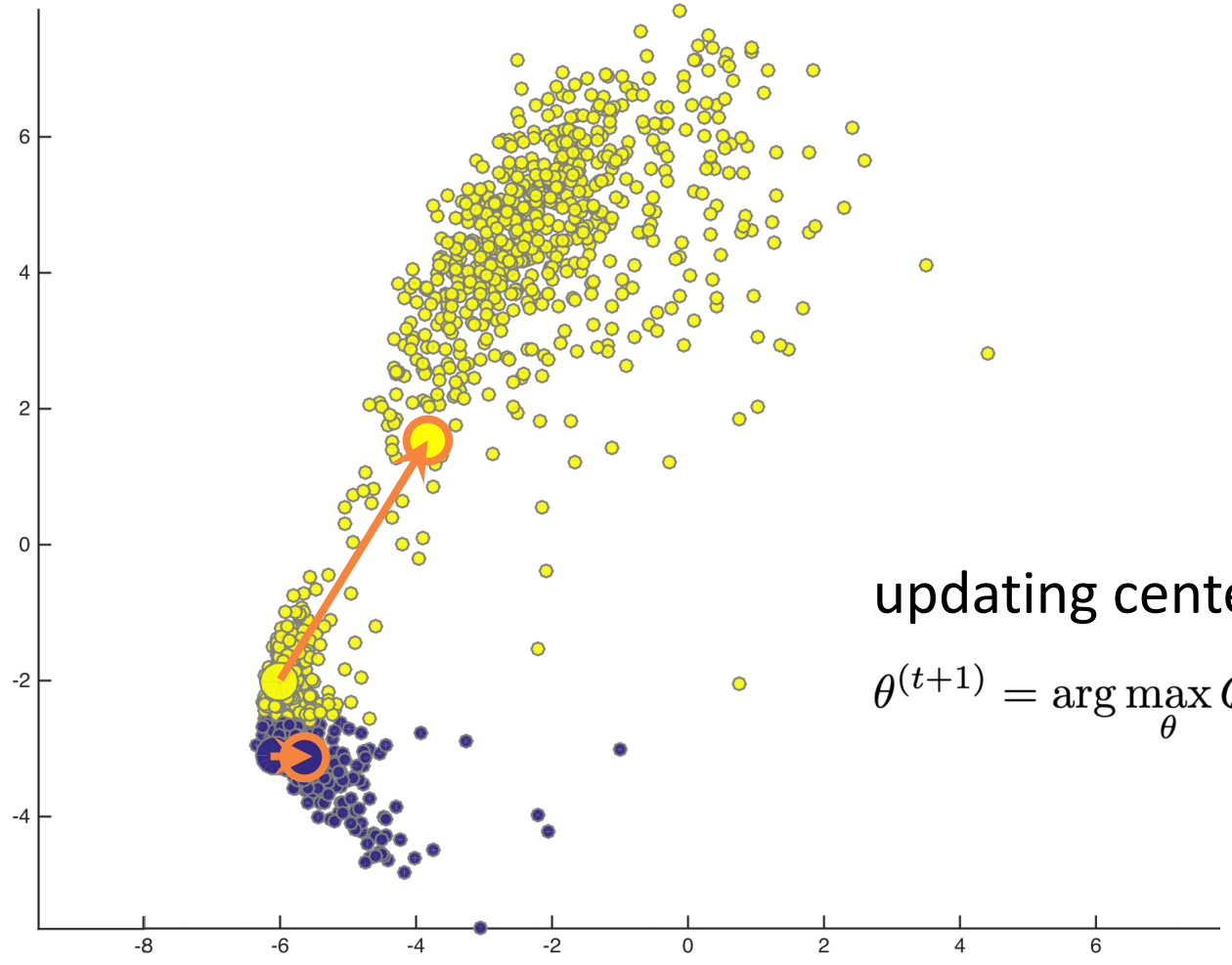


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ
- assignment: E-step
- update: M-step



updating centers by:

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta|\theta^{(t)})$$

Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

A running example of EM: K-means

- cluster centers: θ
- assignment: E-step
- update: M-step

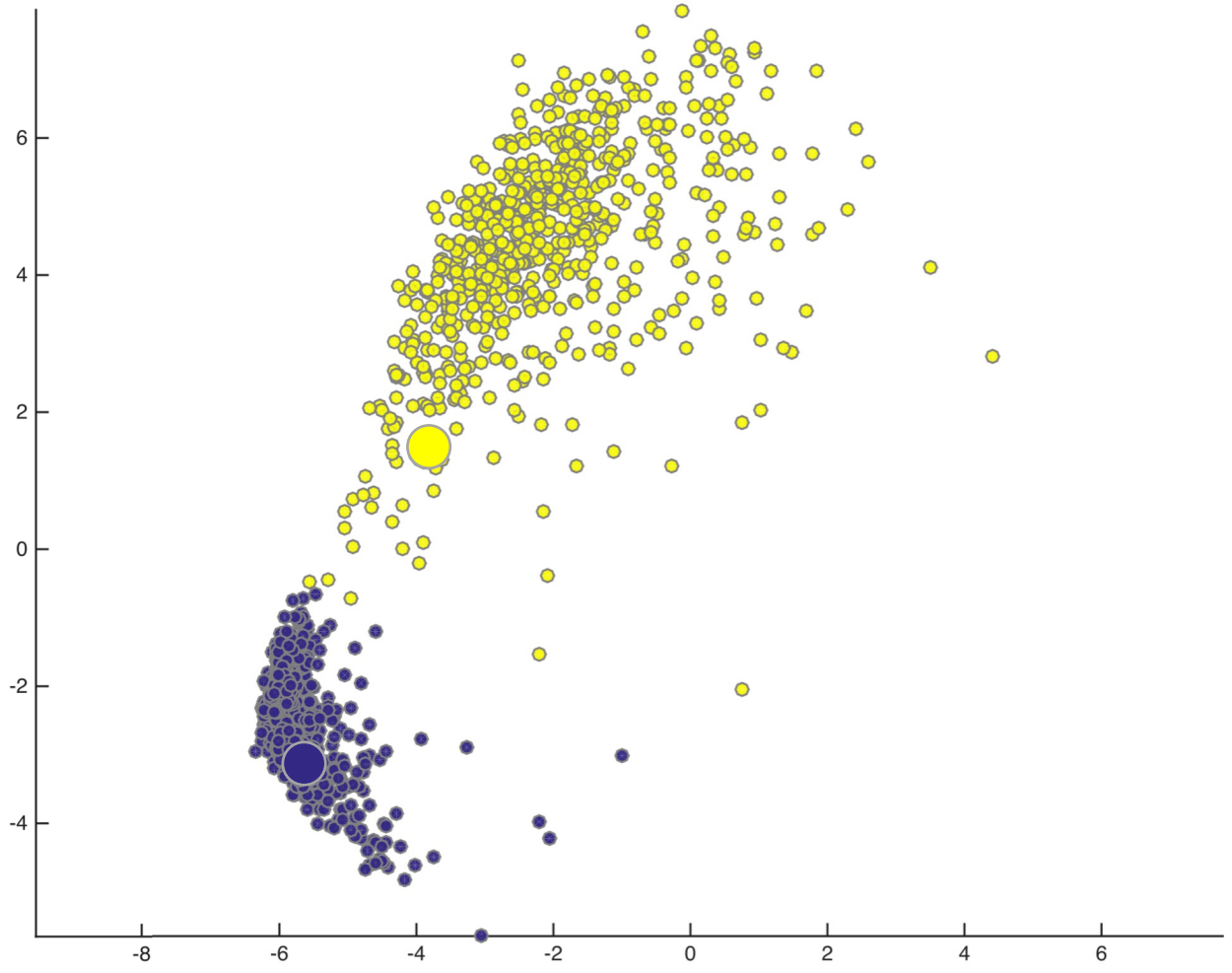
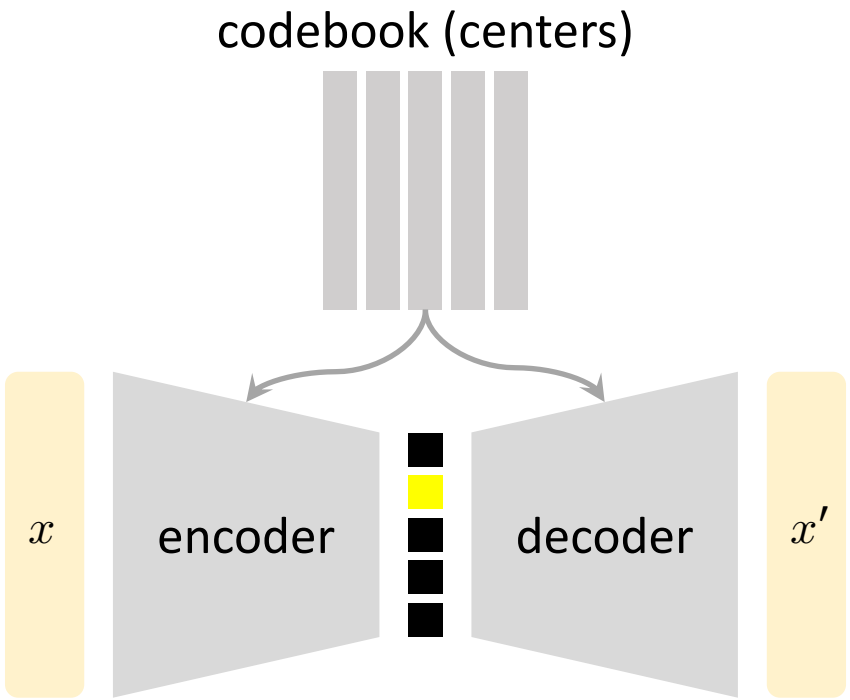
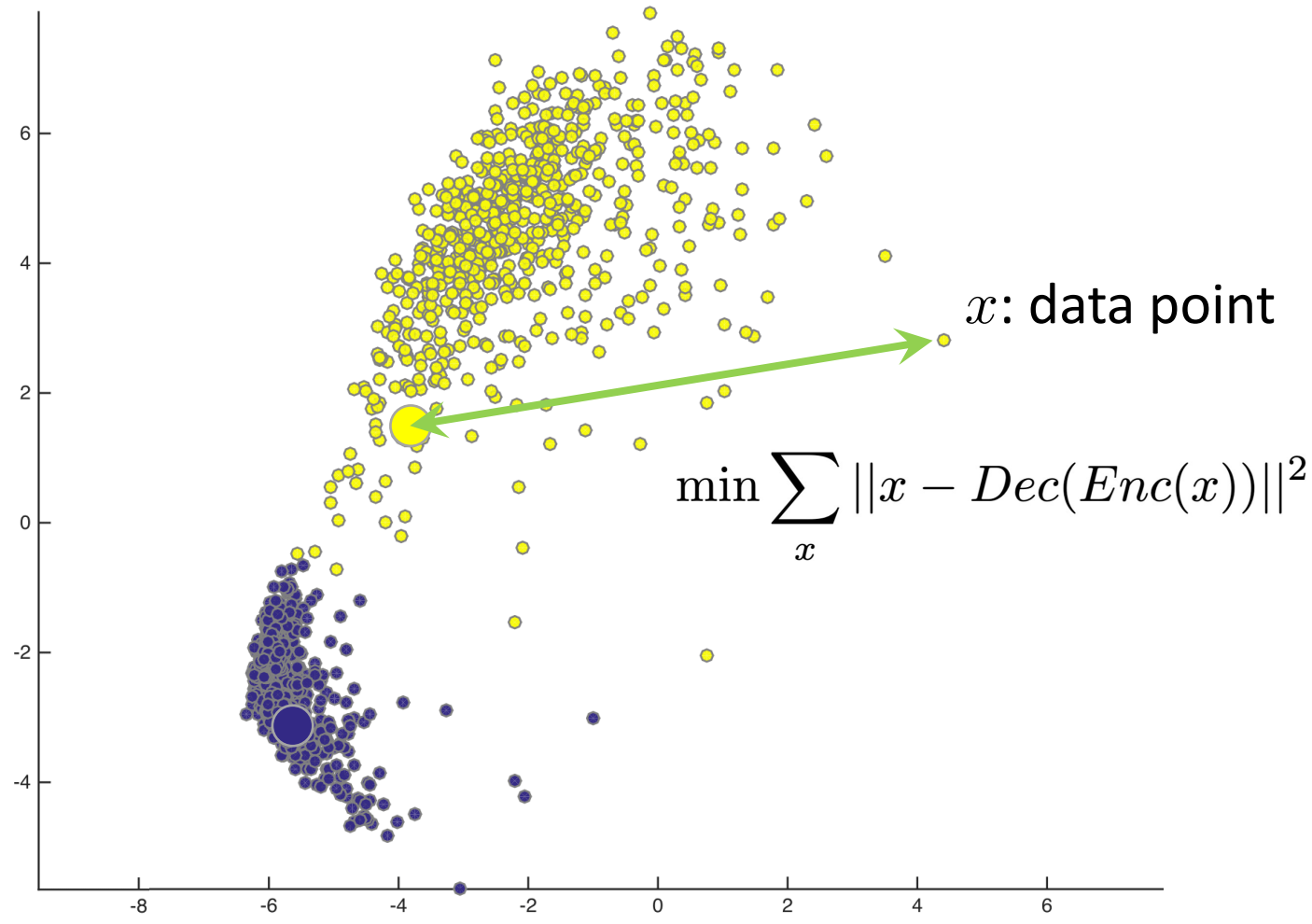


Figure adapted from: Marion Neumann, https://www.cse.wustl.edu/~m.neumann/sp2016/cse517/lecturenotes/lecturenote20.html

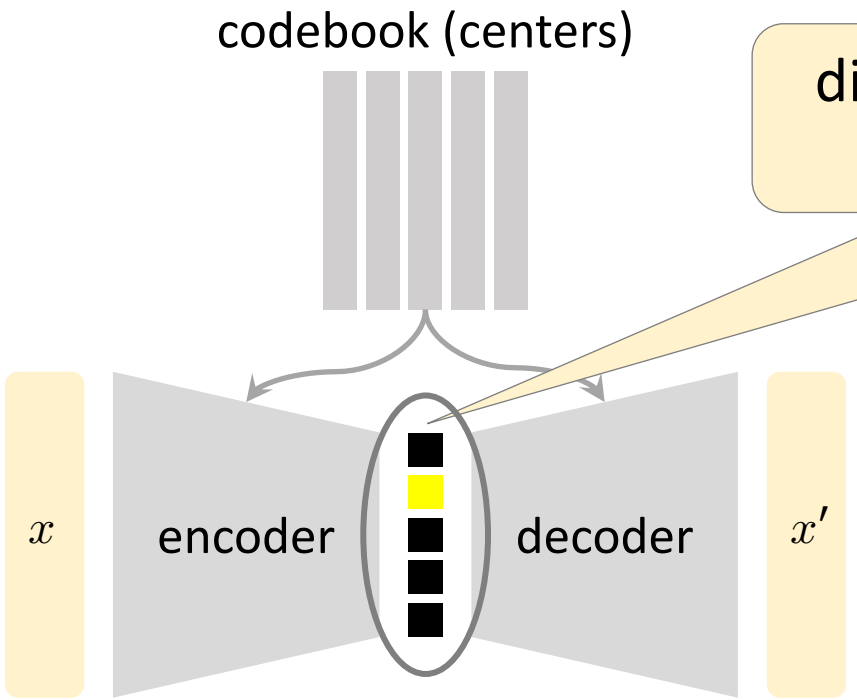
K-means as **Autoencoder**



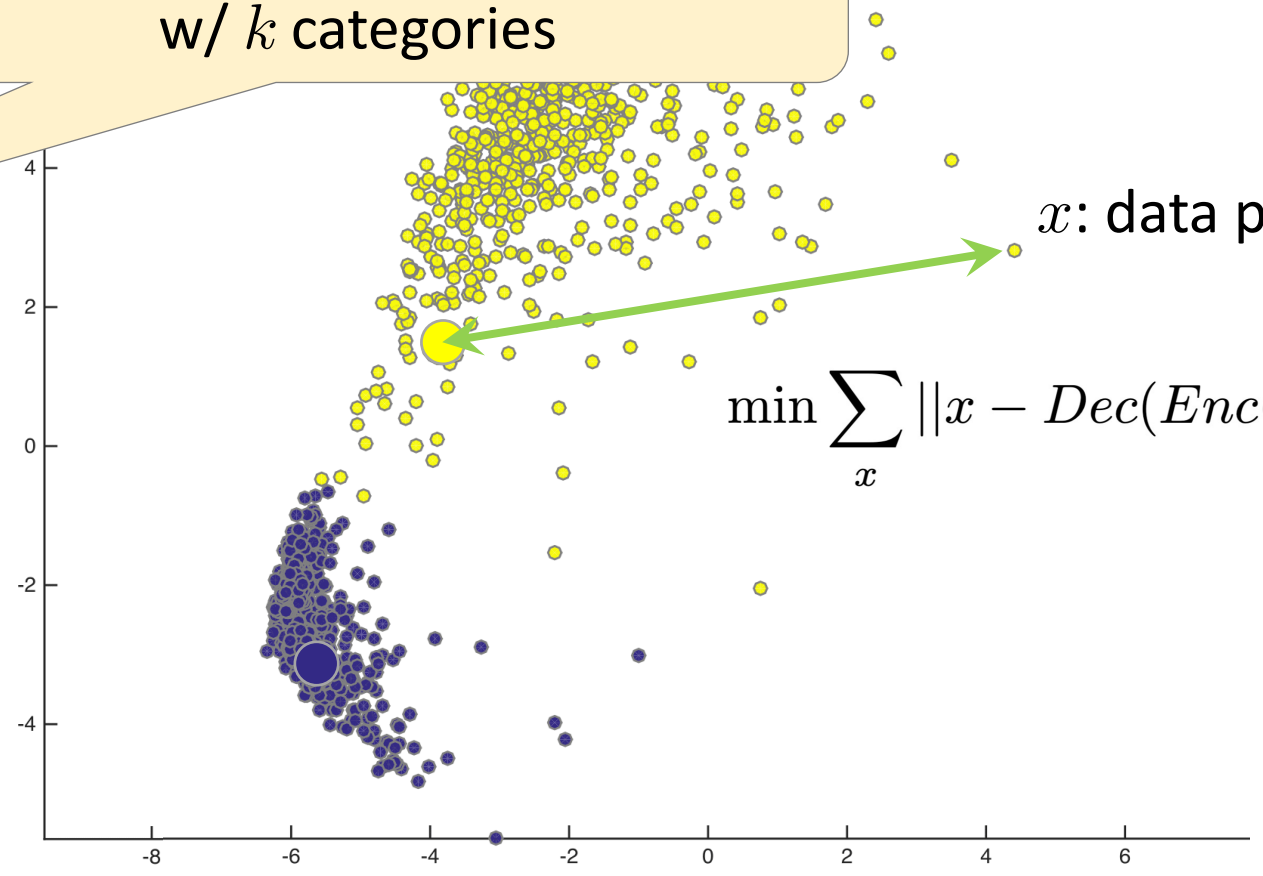
- encode: map x to one-hot
- decode: map one-hot to x'
- x' is a center



K-means as **Autoencoder**



discrete (categorical) distribution
w/ k categories

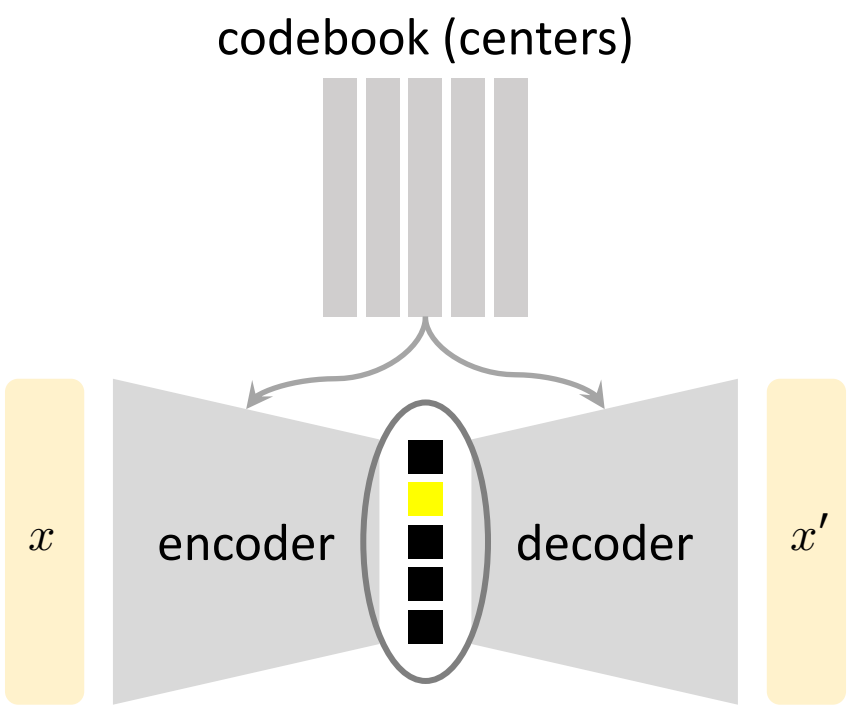


x: data point

$$\min \sum_x ||x - Dec(Enc(x))||^2$$

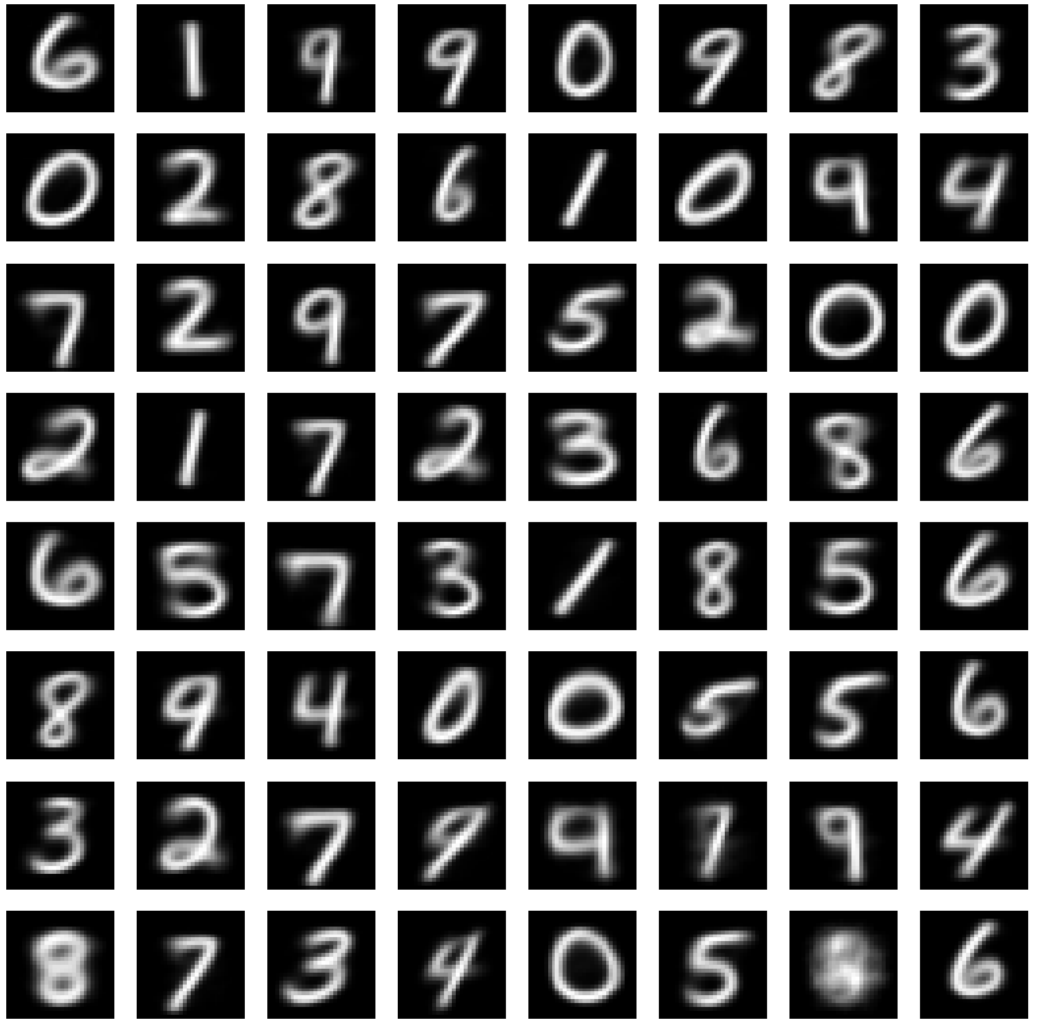
- encode: map x to one-hot
- decode: map one-hot to x'
- x' is a center

K-means as **Autoencoder**



- encode: map x to one-hot
- decode: map one-hot to x'
- x' is a center

codebook on MNIST, k = 64

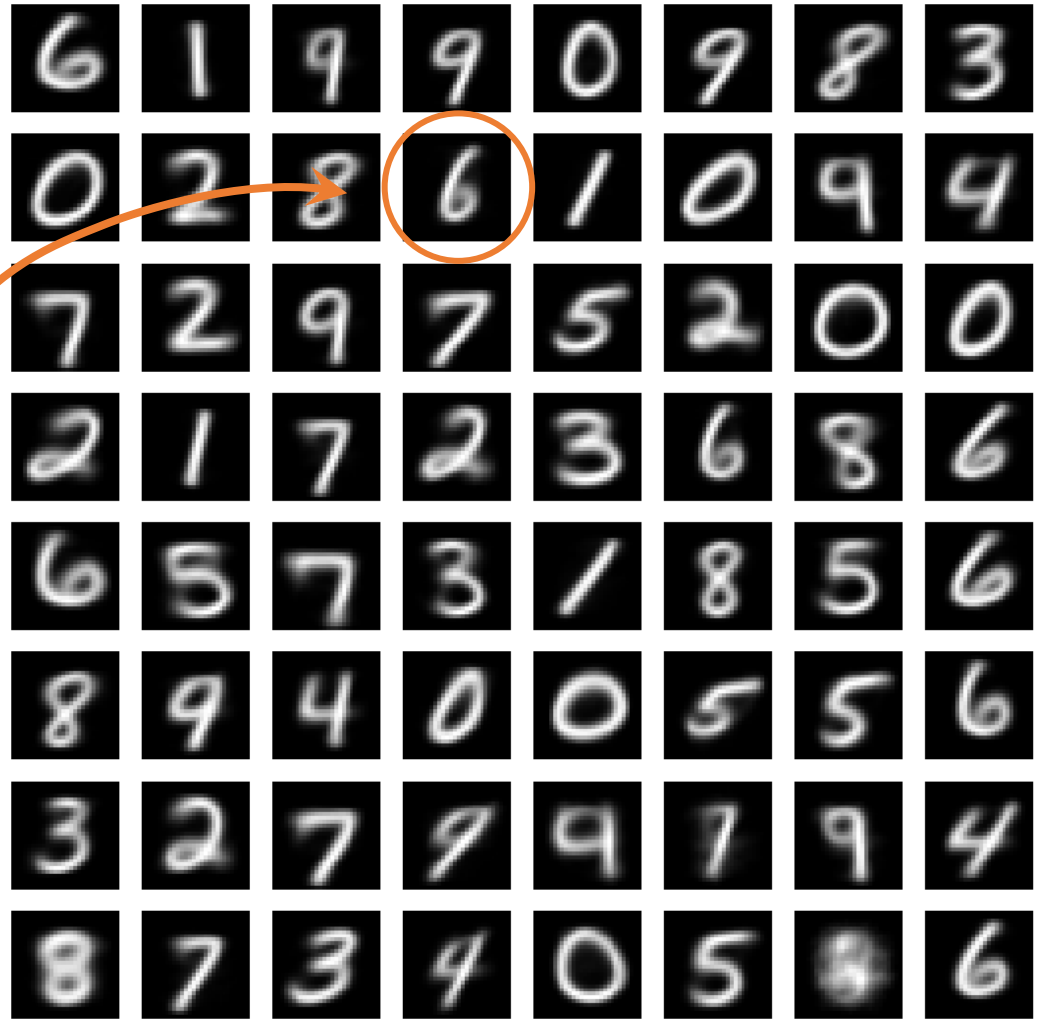


K-means as **Generative Models**

- randomly sample: $z \sim \mathcal{U}[0, k)$
- map z by the decoder
- generation result is one codeword

$z = 11$

codebook on MNIST, k = 64



- this is a valid generative model
- but not a “good” one
- but a good thought model

thus far, ...

- **VAE**: maximize ELBO
  - parameterize q by network
  - optimize by Stochastic Gradient Descent

- **EM**: maximize ELBO
  - parameterize q analytically
  - optimize by Coordinate Descent

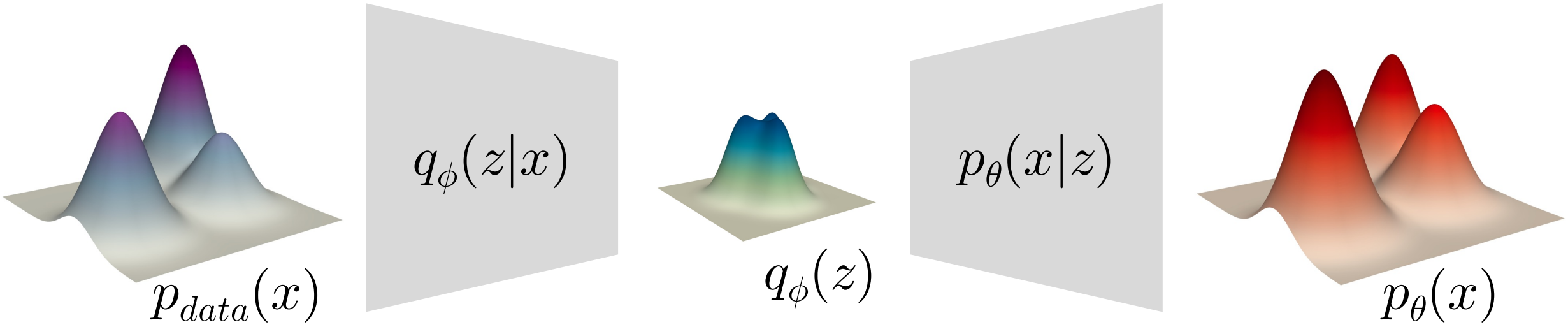
- **K-means**:
  - special case of EM; special case of AE
  - discrete distribution

- next: **VQ-VAE**

Vector Quantized VAE (VQ-VAE)

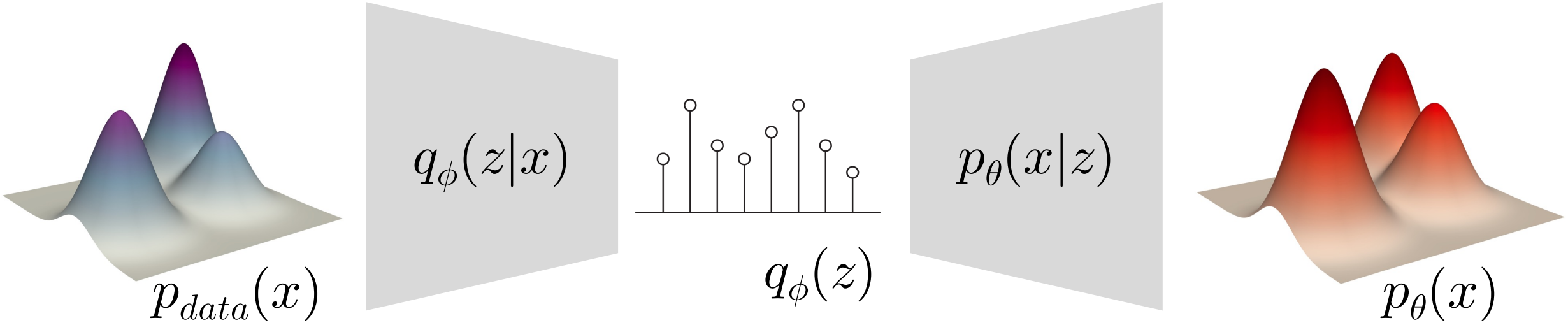
Recap

- Original VAE: latent variables are continuous



Discrete Latent Variables

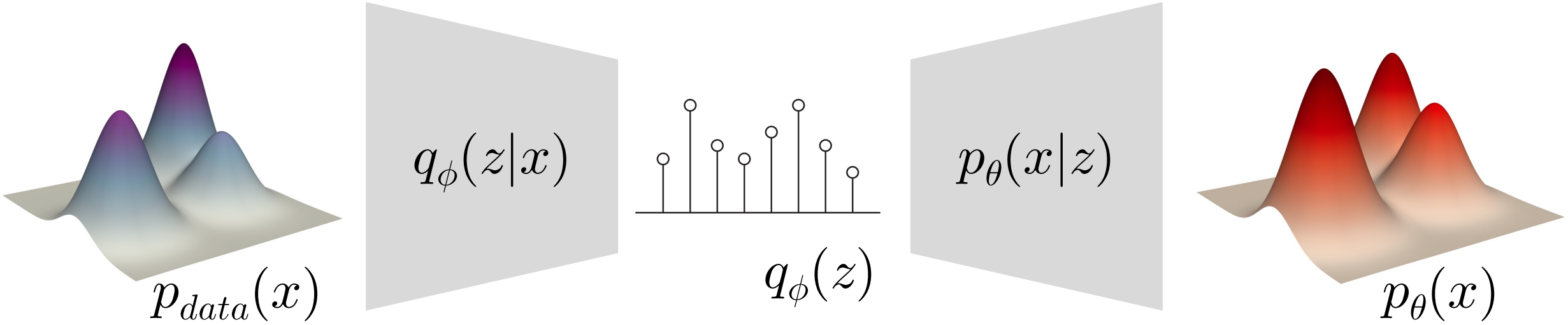
- model **multimodal** distributions
- **categorical**: no particular relation between numbers (SSN, zip code, ...)
- **symbolic**: language, speech, planning, ...



Discrete Latent Variables + VAE

Maximize ELBO

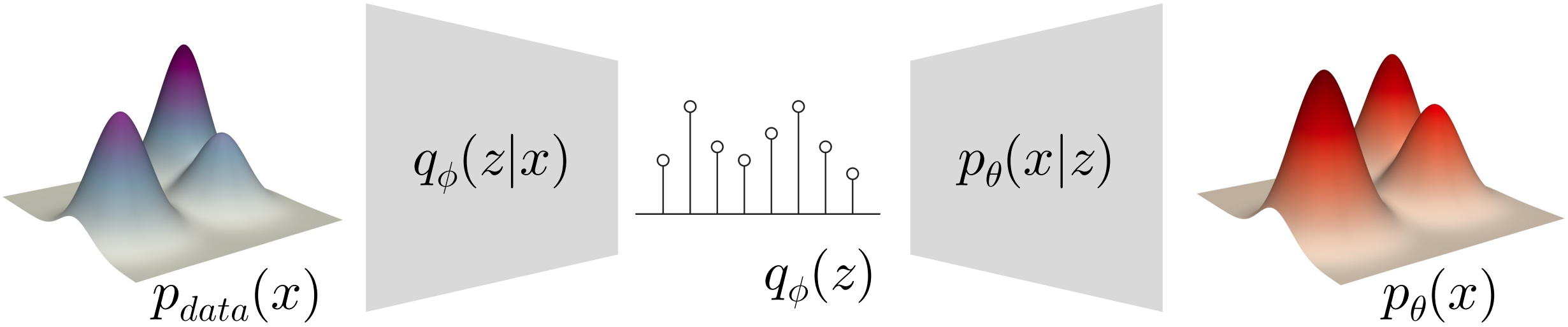
- Reconstruction loss: about x
- Regularization loss: about z (discrete)



Discrete Latent Variables + VAE

Reconstruction loss: about x

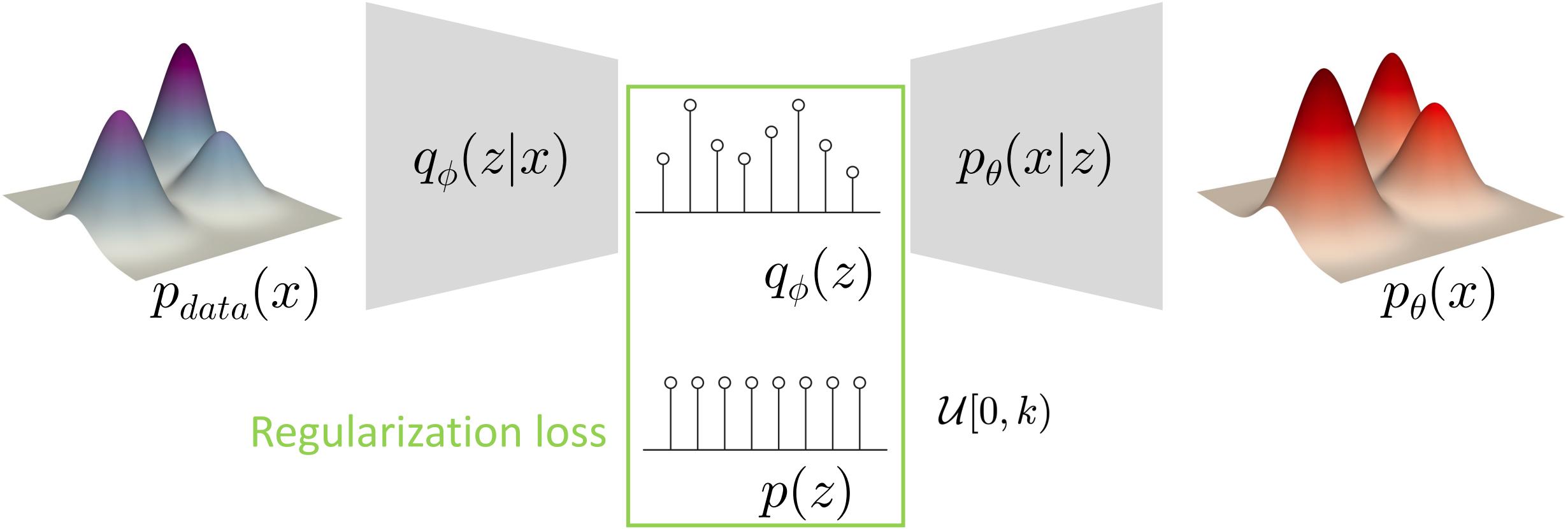
- same as VAE: $-\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big]$



Discrete Latent Variables + VAE

Regularization loss: about z

- conceptually, same as VAE: $\mathcal{D}_{\mathrm{KL}}\Big(q_\phi(z|x)||p(z)\Big)$
- but how can we backprop w.r.t. discrete sampling?



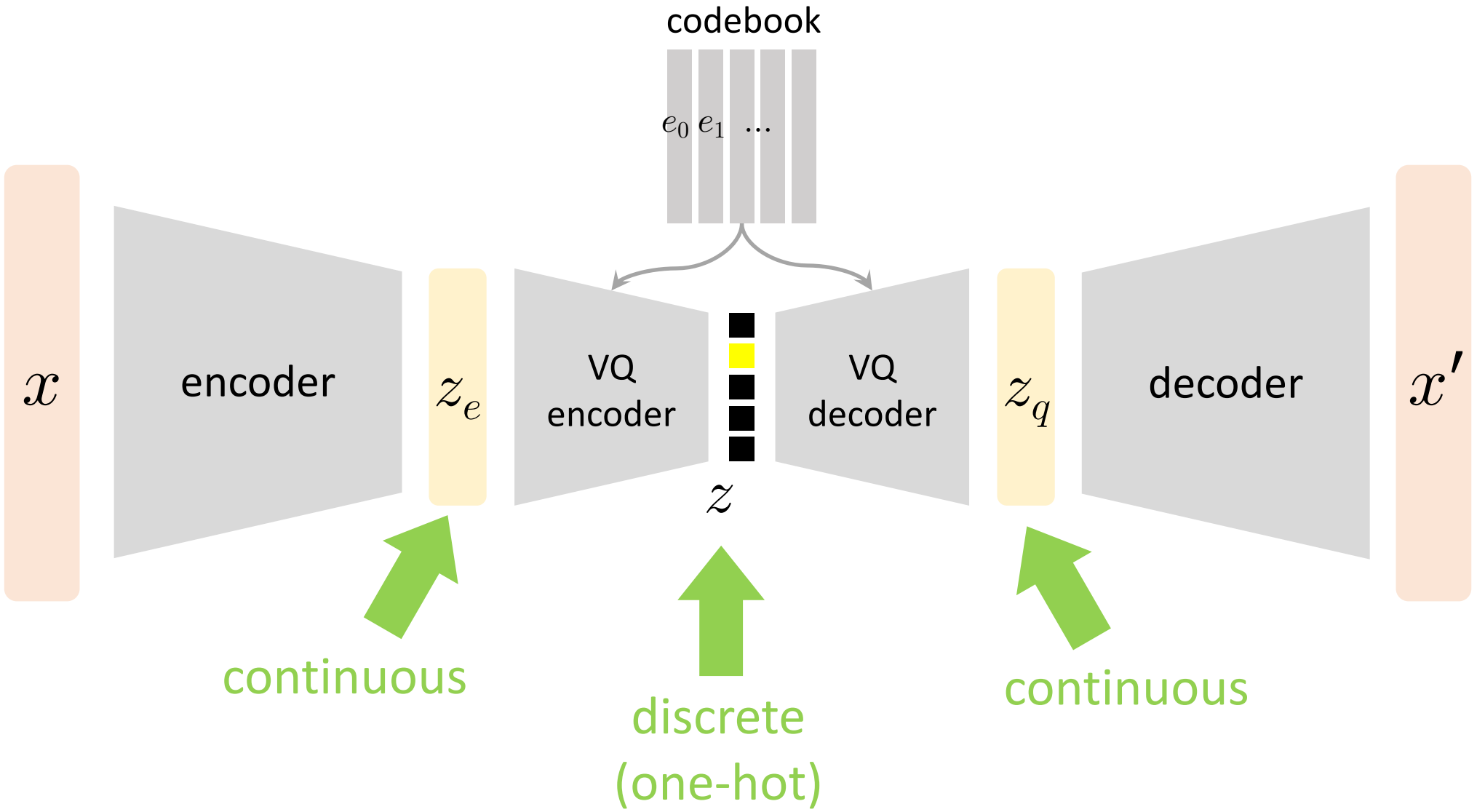
Discrete Latent Variables + VAE

Solution: K-means

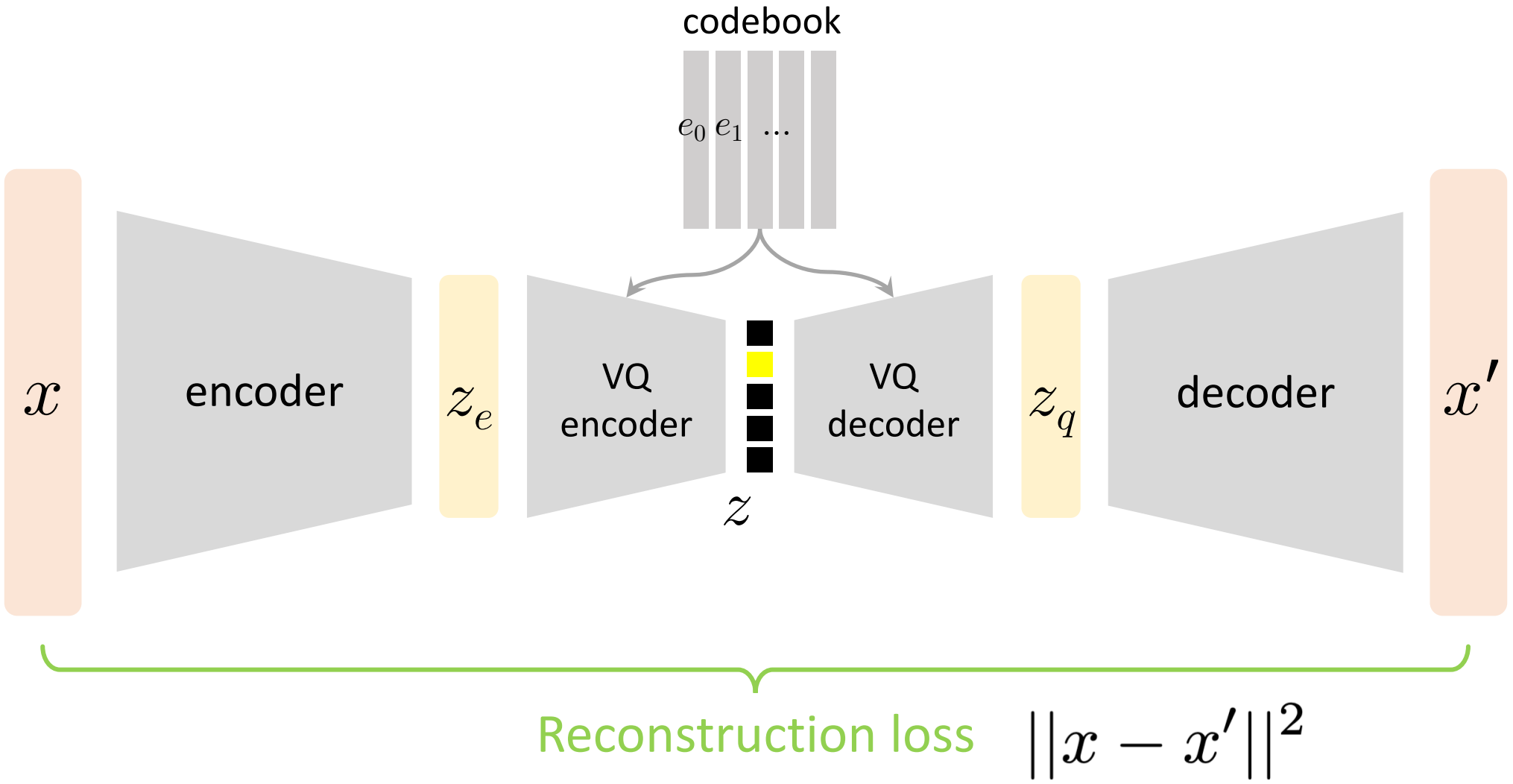
- K-means is autoencoding
- K-means has an objective function (reconstruction loss)
- K-means implicitly encourages codebook uniformity

This leads us to VQ-VAE ...

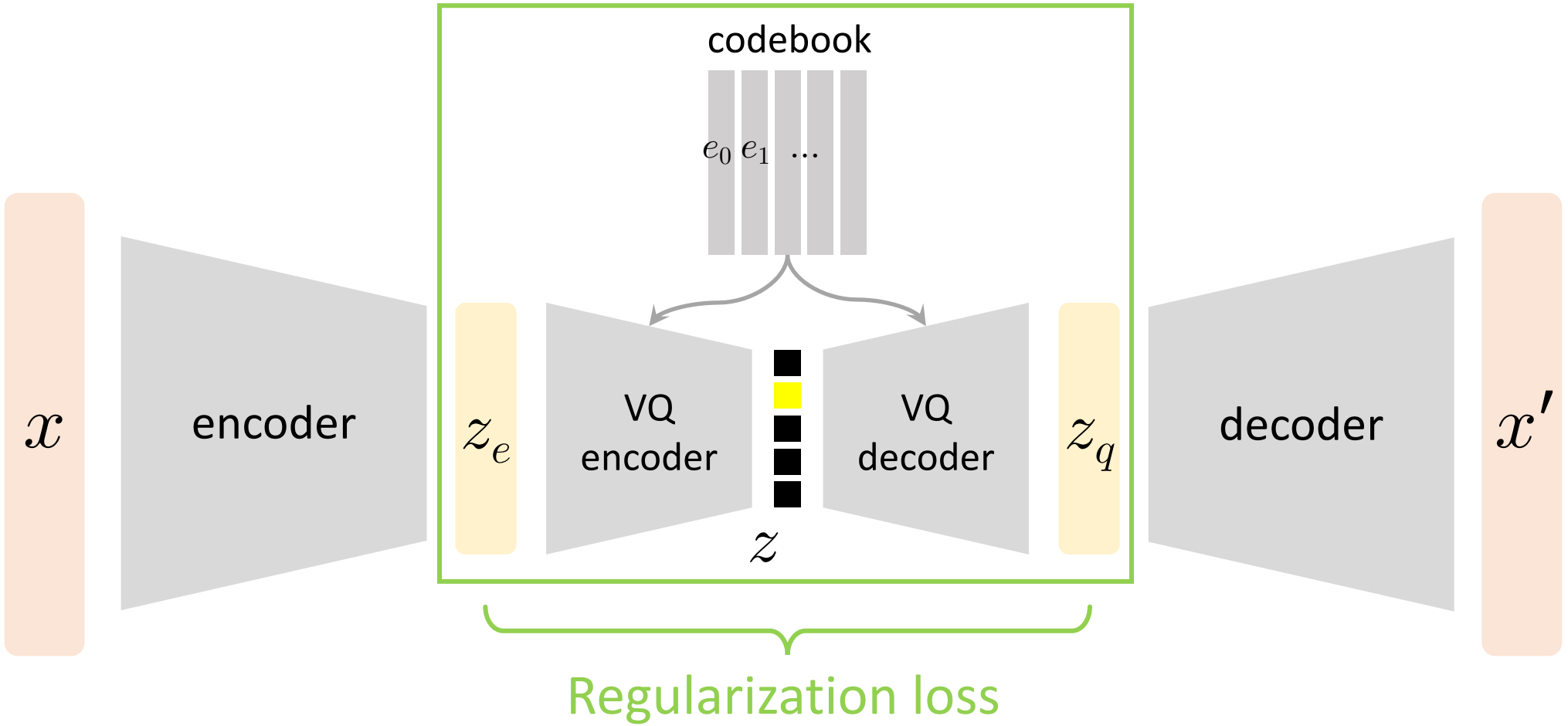
Vector Quantized VAE



Vector Quantized VAE



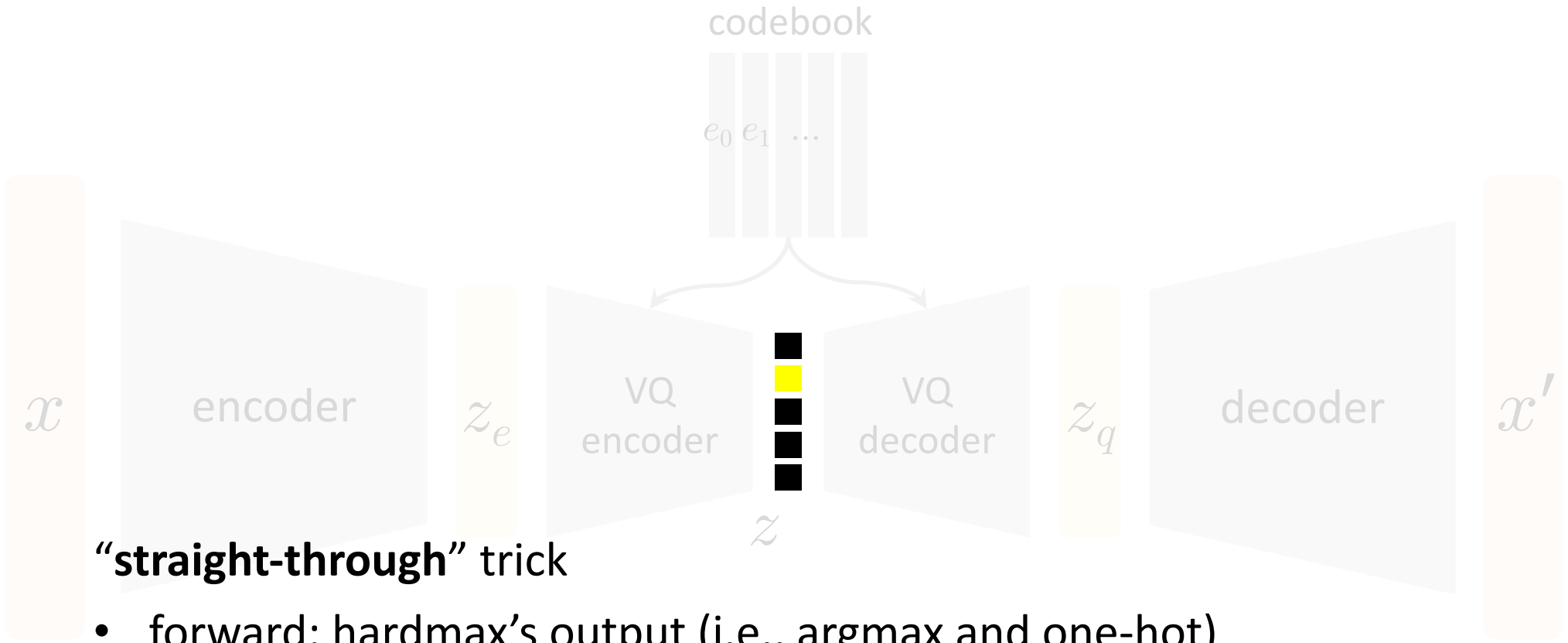
Vector Quantized VAE



conceptually, this is the K-means reconstruction loss: $\|z_e - z_q\|^2$

*The VQ-VAE paper uses $\|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$ which weights the gradients differently

How to backprop through one-hot vector?



"**straight-through**" trick

- forward: hardmax's output (i.e., argmax and one-hot)
- backward: softmax's gradient
- in code: `stop_grad(hardmax(y) – softmax(y)) + softmax(y)`

Vector Quantized VAE

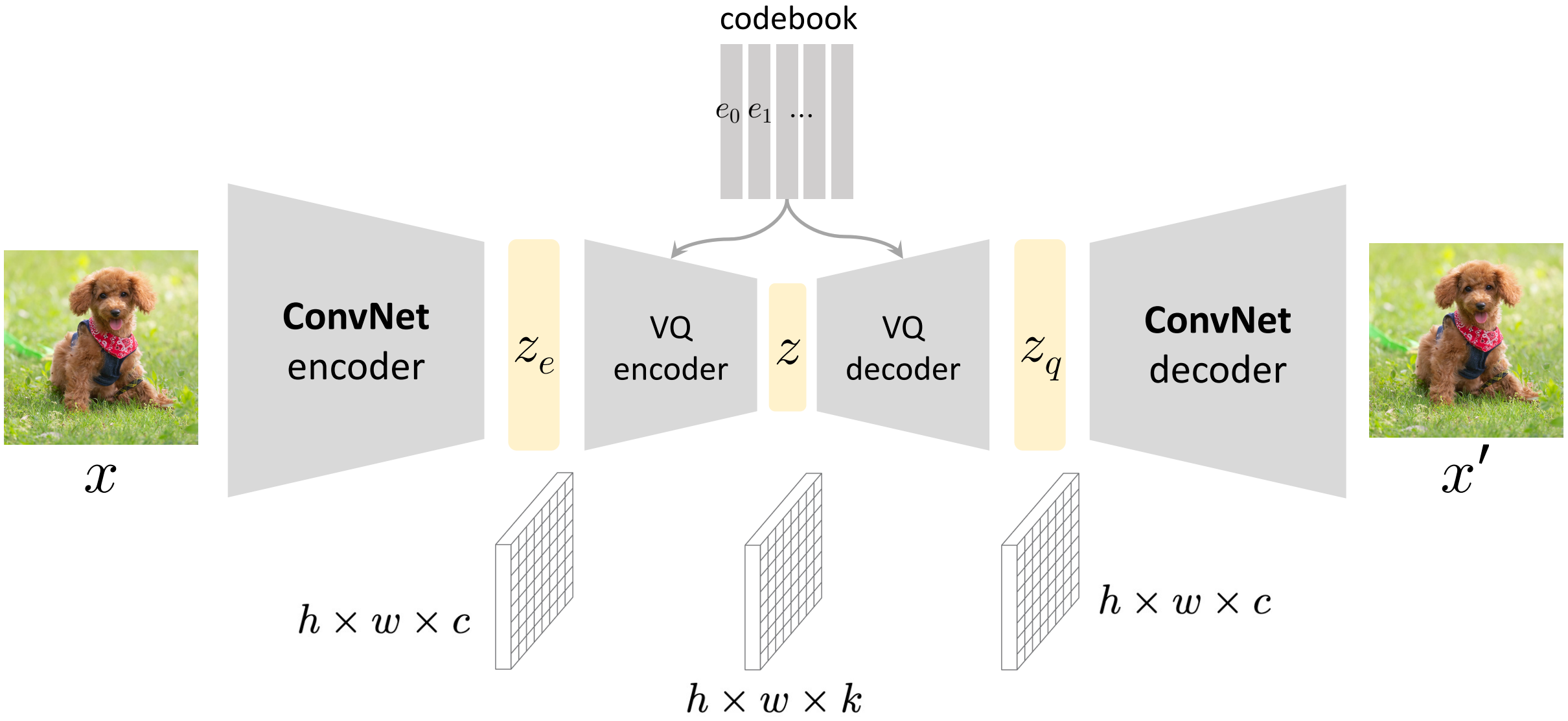
A single one-hot latent is not useful

- it's "deep K-means": with deep encoder/decoder
- a valid generative model; but not a "good" one

VQ-VAE: often used as "**tokenizers**"

- output multiple one-hot vectors
- don't reduce latent spatial/temporal size to 1
- use ConvNet/Transformer as encoder and decoder

VQ-VAE as Tokenizers



Notes

- Both VAE and VQ-VAE can be “tokenizers” (produce spatial latents).

But:

- prior $p(z)$ only models per-token (per-location) distribution
- prior $p(z)$ doesn’t model **joint** distribution across tokens
- spatial tokens are not **independent**
- at inference, we can’t sample from **i.i.d.** prior $p(z)$

Next: modeling joint distribution:

- Autoregressive models
- Masked models
- Diffusion models

This Lecture

- Variational Autoencoder (VAE)
- Relation to Expectation-Maximization (EM)
- Vector Quantized VAE (VQ-VAE)

Main References

- Kingma and Welling. “Auto-Encoding Variational Bayes”, ICLR 2014
- Neal and Hinton. “A view of the EM algorithm that justifies incremental, sparse, and other variants”, 1999
- Hastie, et al. “The Elements of Statistical Learning”, 2001
- van den Oord, et al. “Neural Discrete Representation Learning”, NeurIPS 2017